

SQL และฐานข้อมูล

คู่มือฉบับรวม 100 หน้า

จากพื้นฐาน SQL สู่การออกแบบ ความปลอดภัย
ประสิทธิภาพ และการดูแลฐานข้อมูล

หน้า 2-73 คงต้นฉบับผู้ใช้ 72 หน้าไว้ครบ เลขหน้าที่หัวกระดาษเป็นเลขต้นฉบับ

หน้า 74-99 ภาคเสริม 26 หัวข้อจากเอกสารทางการ พร้อมแบบทบทวนและตัวอย่างสาธิต

หน้า 100 บรรณานุกรม · ไฟล์ Markdown ใช้เลขหน้าของฉบับรวมนี้ในการอ้างอิง

แผนที่ภาคเสริม

74 / CTE WITH (คิวรีมีชื่อ)

75 / Recursive CTE (คิวรีเรียกซ้ำ)

76 / Window Functions ROW_NUMBER OVER PARTITION BY (ฟังก์ชัน
หน้าต่าง)

77 / Transaction Isolation MVCC (ระดับการแยกธุรกรรม)

78 / SQL Injection Prepared Statements (ป้องกันการแทรกคำสั่ง)

79 / Normalization 1NF 2NF 3NF (การออกแบบลดความซ้ำซ้อน)

80 / EXPLAIN Query Plan (วิเคราะห์แผนคิวรี)

81 / Index Performance (ประสิทธิภาพดัชนี)

82 / Partitioning (แบ่งตาราง)

83 / JSON JSONB (ข้อมูลกึ่งโครงสร้าง)

84 / Backup Restore PITR (สำรองและกู้คืน)

85 / Replication Standby (ทำสำเนาฐานข้อมูล)

86 / Row Level Security RLS (สิทธิ์ระดับแถว)

87 / CREATE VIEW (ตารางเสมือน)

88 / Identity AUTO INCREMENT (เลขอัตโนมัติ)

89 / Subquery EXISTS NOT IN NULL (คิวรีย่อยและค่าว่าง)

90 / Transaction BEGIN COMMIT ROLLBACK SAVEPOINT (ธุรกรรมและ
จุดย้อนกลับ)

91 / Locking Deadlock (การล็อกและเดดล็อก)

92 / Roles GRANT REVOKE (บทบาทและสิทธิ์)

93 / Date Time Timestamp Timezone (วันเวลาและเขตเวลา)

94 / Materialized View REFRESH (ผลลัพธ์ที่จัดเก็บไว้)

95 / Trigger BEFORE AFTER (การทำงานตามเหตุการณ์)

96 / UPSERT ON CONFLICT (เพิ่มหรือปรับข้อมูลเมื่อชนคีย์)

97 / NoSQL Document Modeling (ออกแบบฐานข้อมูลเอกสาร)

98 / Sharding Shard Key (กระจายข้อมูลข้ามเครื่อง)

99 / Schema Migration ALTER TABLE (เปลี่ยนโครงสร้างข้อมูล)

เนื้อหาครอบคลุมหัวข้อหลัก ไม่ใช่ทุกคุณสมบัติของทุก DBMS ข้อความเดิมและไวยากรณ์เฉพาะระบบต้องพิจารณาตามบริบท ตัวอย่างใหม่เป็นแบบฝึกหัดในฐานทดลอง

คู่มือภาษา SQL เบื้องต้น

เรียบเรียงจากเนื้อหาบทเรียน SQL

อ้างอิงจาก: <https://www.w3schools.com/sql/>

บทนำ

SQL (Structured Query Language)

เป็นภาษามาตรฐานที่ใช้สำหรับเข้าถึงและจัดการข้อมูลในฐานข้อมูลเชิงสัมพันธ์ (Relational Database) เอกสารฉบับนี้เรียบเรียงเนื้อหาพื้นฐานของภาษา SQL

โดยครอบคลุมตั้งแต่แนวคิดพื้นฐาน คำสั่งดึงข้อมูล การกรองข้อมูล การเรียงลำดับ การเพิ่ม-แก้ไข-ลบข้อมูล ไปจนถึงฟังก์ชันรวมข้อมูลที่ใช้งานบ่อย

เพื่อเป็นแนวทางสำหรับผู้เริ่มต้นศึกษาการเขียนคำสั่ง SQL

ตัวอย่างคำสั่งทั้งหมดในเอกสารนี้อ้างอิงจากฐานข้อมูลตัวอย่างที่มีตาราง Customers (ข้อมูลลูกค้า), Products (ข้อมูลสินค้า), Orders / OrderDetails (ข้อมูลคำสั่งซื้อ) และ Employees (ข้อมูลพนักงาน) ซึ่งเป็นฐานข้อมูลตัวอย่างมาตรฐานที่ใช้ประกอบการสอน SQL ทั่วไป

1. SQL คืออะไร

SQL ย่อมาจาก Structured Query Language

เป็นภาษามาตรฐานสำหรับติดต่อกับระบบจัดการฐานข้อมูลเชิงสัมพันธ์ (RDBMS - Relational Database Management System) ไม่ว่าจะเป็น MySQL, Microsoft SQL Server, Oracle, PostgreSQL หรือ Microsoft Access

SQL ทำอะไรได้บ้าง

- ค้นหาและดึงข้อมูล (Query) จากฐานข้อมูล
- เพิ่ม แก้ไข และลบข้อมูลในตาราง
- สร้าง แก้ไข และลบตารางหรือฐานข้อมูล
- กำหนดสิทธิ์การเข้าถึงตารางและมุมมองข้อมูล (View)

1. ฐานข้อมูล (Database) คืออะไร

ฐานข้อมูล (Database) คือชุดข้อมูลที่มีโครงสร้าง ถูกจัดเก็บอย่างเป็นระบบในคอมพิวเตอร์ และควบคุมดูแลโดยซอฟต์แวร์ที่เรียกว่าระบบจัดการฐานข้อมูล (Database Management System หรือ DBMS) ข้อมูล ซอฟต์แวร์ DBMS และแอปพลิเคชันที่เกี่ยวข้อง เมื่อรวมกันจะเรียกว่า “ระบบฐานข้อมูล” (Database System)

ข้อมูลในฐานข้อมูลส่วนใหญ่ในปัจจุบันจะถูกจัดเก็บในรูปแบบตาราง (Table)

ที่ประกอบด้วยแถวและคอลัมน์

เพื่อให้สามารถประมวลผลและค้นหาข้อมูลได้อย่างมีประสิทธิภาพ และสามารถเข้าถึง จัดการ แก้ไข ปรับปรุง ควบคุม และจัดระเบียบข้อมูลได้โดยง่าย ฐานข้อมูลส่วนใหญ่ใช้ภาษา SQL (Structured Query Language) เป็นเครื่องมือหลักในการเขียนและค้นหาข้อมูล

2. RDBMS และแนวคิดตาราง

RDBMS ย่อมาจาก Relational Database Management System

เป็นโปรแกรมที่ใช้ดูแลรักษาฐานข้อมูลเชิงสัมพันธ์

และเป็นพื้นฐานของระบบฐานข้อมูลสมัยใหม่แทบทั้งหมด เช่น MySQL, Microsoft SQL Server,

Oracle Database และ Microsoft Access ข้อมูลใน RDBMS

จะถูกจัดเก็บในอ็อบเจกต์ที่เรียกว่า “ตาราง” (Table) ซึ่งเป็นชุดของข้อมูลที่เกี่ยวข้องกัน

ประกอบด้วย:

- **คอลัมน์ (Column / Field):** เป็นแนวตั้งของตาราง ใช้เก็บข้อมูลชุดหนึ่งที่มีลักษณะประเภทเดียวกัน
- **แถว (Row / Record):** เป็นแนวนอนของตาราง ใช้เก็บข้อมูลแต่ละรายการในตาราง

ตัวอย่างเช่น ตาราง “Customers” จากฐานข้อมูลตัวอย่าง Northwind มีคอลัมน์ ได้แก่

CustomerID, CustomerName, ContactName, Address, City, PostalCode และ Country

โดยมีข้อมูลทั้งหมด 5 แถว หรือ 5 รายการลูกค้า

3. ฐานข้อมูลเชิงสัมพันธ์ (Relational Database)

ฐานข้อมูลเชิงสัมพันธ์กำหนดความสัมพันธ์ของข้อมูลในรูปแบบของตารางที่เชื่อมโยงกัน

โดยอาศัยข้อมูลที่มีร่วมกันระหว่างตาราง (Common data) เช่น ในฐานข้อมูล Northwind

ที่มีตาราง Customers, Orders และ Shippers:

- ความสัมพันธ์ระหว่างตาราง Customers และ Orders คือ คอลัมน์ CustomerID
- ความสัมพันธ์ระหว่างตาราง Orders และ Shippers คือ คอลัมน์ ShipperID

การเชื่อมโยงข้อมูลลักษณะนี้อาศัยหลักการของ Primary Key และ Foreign Key

ซึ่งเป็นตัวระบุเฉพาะ (Unique identifier) ที่แสดงความสัมพันธ์ระหว่างตารางต่าง ๆ

ทำให้สามารถรวมข้อมูลจากหลายตารางเข้าด้วยกันเพื่อสร้างรายงานที่มีคุณค่าได้ เช่น

รายงานยอดขายแยกตามอุตสาหกรรมหรือบริษัท

4. ภาษา SQL คืออะไร

SQL (Structured Query Language)

เป็นภาษาโปรแกรมมาตรฐานที่ใช้ในระบบจัดการฐานข้อมูลเชิงสัมพันธ์เกือบทั้งหมด สำหรับค้นหา (Query) จัดการ (Manipulate) และกำหนดโครงสร้างข้อมูล (Define)

รวมถึงควบคุมสิทธิ์การเข้าถึงข้อมูล เนื่องจากฐานข้อมูลเชิงสัมพันธ์เกี่ยวข้องกับอย่างใกล้ชิดกับ SQL จึงมักถูกเรียกว่า “ฐานข้อมูล SQL” เช่นกัน

SQL เริ่มพัฒนาขึ้นที่ IBM ในช่วงทศวรรษ 1970 โดยมี Oracle เป็นผู้มีส่วนร่วมสำคัญ ซึ่งนำไปสู่การกำหนดมาตรฐาน SQL ของ ANSI และมีการต่อยอดพัฒนาโดยบริษัทต่าง ๆ เช่น IBM, Oracle และ Microsoft แม้ปัจจุบันจะมีภาษาโปรแกรมใหม่ ๆ เกิดขึ้น แต่ SQL ก็ยังคงถูกใช้งานอย่างแพร่หลาย

5. ประวัติความเป็นมาของฐานข้อมูลเชิงสัมพันธ์และ SQL

ก่อนจะมีฐานข้อมูลเชิงสัมพันธ์ องค์กรต่าง ๆ ใช้ระบบฐานข้อมูลแบบลำดับชั้น (Hierarchical Database) ที่มีโครงสร้างคล้ายต้นไม้

ซึ่งมีข้อจำกัดด้านความยืดหยุ่นและมักผูกติดกับแอปพลิเคชันเฉพาะ

- ปี ค.ศ. 1970 นักวิจัยของ IBM ชื่อ Edgar F. Codd ตีพิมพ์บทความ “A Relational Model of Data for Large Shared Data Banks”
ซึ่งเป็นการเสนอแนวคิดโมเดลฐานข้อมูลเชิงสัมพันธ์เป็นครั้งแรก
โดยเสนอให้จัดเรียงข้อมูลตามความสัมพันธ์ที่มีความหมาย ในรูปแบบคู่ attribute-value (tuple)
- ปี ค.ศ. 1973 IBM เริ่มโครงการ System R ที่ San Jose Research Laboratory
(ปัจจุบันคือ Almaden Research Center)

เพื่อพิสูจน์ทฤษฎีเชิงสัมพันธ์ในระดับอุตสาหกรรม และกลายเป็นพื้นที่ทดสอบภาษา SQL จนได้รับการยอมรับอย่างแพร่หลาย

- ภาษา SQL ถูกคิดค้นโดย Don Chamberlin และ Ray Boyce แห่ง IBM เดิมเรียกว่า “SEQUEL” แต่ภายหลังเปลี่ยนชื่อเป็น “SQL” เนื่องจากปัญหาด้านเครื่องหมายการค้า
- ปี ค.ศ. 1983 IBM เปิดตัวตระกูลผลิตภัณฑ์ DB2
ซึ่งเป็นฐานข้อมูลเชิงสัมพันธ์ตระกูลที่สองของ IBM
และยังคงเป็นหนึ่งในผลิตภัณฑ์ที่ประสบความสำเร็จที่สุด
รองรับธุรกรรมนับพันล้านรายการต่อวันบนโครงสร้างพื้นฐานคลาวด์จนถึงปัจจุบัน

6. วิวัฒนาการของฐานข้อมูล

- ทศวรรษ 1960: ฐานข้อมูลแบบ Navigational เช่น Hierarchical Database (โครงสร้างต้นไม้ รองรับความสัมพันธ์แบบ one-to-many) และ Network Database (ยืดหยุ่นกว่า รองรับหลายความสัมพันธ์) แต่ยังมีความซับซ้อนและไม่ยืดหยุ่น
- ทศวรรษ 1980: ฐานข้อมูลเชิงสัมพันธ์ (Relational Database)
ได้รับความนิยมอย่างกว้างขวาง
- ทศวรรษ 1990: ฐานข้อมูลเชิงวัตถุ (Object-oriented Database) เริ่มเป็นที่แพร่หลาย
- ต่อมา: ฐานข้อมูลแบบกระจาย (Distributed Database)
เกิดขึ้นเพื่อรองรับการเติบโตของอินเทอร์เน็ต
และความต้องการประมวลผลข้อมูลที่ไม่มีโครงสร้าง (Unstructured data) ได้รวดเร็วขึ้น
- ปัจจุบัน: ฐานข้อมูลบนคลาวด์ (Cloud Database) และฐานข้อมูลอัตโนมัติ (Autonomous Database) กำลังเปลี่ยนรูปแบบการจัดเก็บและบริหารจัดการข้อมูล
โดยฐานข้อมูลอัตโนมัติตัวแรกถูกประกาศเปิดตัวในช่วงปลายปี ค.ศ. 2017

7. ระบบจัดการฐานข้อมูล (DBMS)

DBMS (Database Management System)

คือซอฟต์แวร์ที่ทำหน้าที่เป็นตัวกลางระหว่างฐานข้อมูลกับผู้ใช้งานหรือโปรแกรมต่าง ๆ

ช่วยให้สามารถดึงข้อมูล อัปเดต และจัดการโครงสร้างข้อมูลได้ นอกจากนี้ DBMS

ยังช่วยในการบริหารจัดการด้านต่าง ๆ เช่น การตรวจสอบประสิทธิภาพ การปรับแต่ง (Tuning)

และการสำรองและกู้คืนข้อมูล (Backup & Recovery)

RDBMS คือ DBMS ที่จัดเก็บข้อมูลในรูปแบบตาราง แตกต่างจาก DBMS

ทั่วไปที่อาจจัดเก็บข้อมูลในรูปแบบไฟล์ ตัวอย่าง DBMS/RDBMS ที่ได้รับความนิยม ได้แก่

MySQL, Microsoft SQL Server, Oracle Database, IBM DB2, PostgreSQL, Microsoft

Access และ FileMaker Pro

8. คุณสมบัติ ACID ของธุรกรรม (Transaction)

ฐานข้อมูลเชิงสัมพันธ์มักเกี่ยวข้องกับธุรกรรม (Transaction)

ซึ่งเป็นชุดคำสั่งที่ต้องทำงานร่วมกันแบบครบถ้วนสมบูรณ์ ตัวอย่างที่นิยมใช้อธิบาย คือ

การโอนเงินระหว่างบัญชี

ที่ยอดเงินจะต้องถูกถอนออกจากบัญชีหนึ่งและฝากเข้าอีกบัญชีหนึ่งพร้อมกันเสมอ

ไม่สามารถเกิดขึ้นเพียงบางส่วนได้ โดยธุรกรรมที่ดีจะมีคุณสมบัติสำคัญ 4 ประการ เรียกว่า ACID ดังนี้

คุณสมบัติ	ความหมาย
Atomicity (ความเป็นเอกภาพ)	การเปลี่ยนแปลงข้อมูลทั้งหมดในธุรกรรมเกิดขึ้นครบถ้วน หรือไม่เกิดขึ้นเลยแม้แต่ส่วนเดียว
Consistency (ความสอดคล้อง)	ข้อมูลอยู่ในสถานะที่ถูกต้องตลอดกระบวนการทำงาน รักษาความสมบูรณ์ของข้อมูล

Isolation (ความเป็นอิสระ)	ธุรกรรมที่ทำงานพร้อมกันจะไม่เห็นสถานะกลางของกันและกัน เสมือนทำงานเรียงลำดับกัน
Durability (ความคงทน)	เมื่อธุรกรรมเสร็จสมบูรณ์ ข้อมูลจะถูกบันทึกถาวร แม้ระบบจะล้มเหลวในภายหลัง

9. ประเภทของฐานข้อมูล

9.1 ฐานข้อมูลเชิงสัมพันธ์ (Relational Database)

จัดเก็บข้อมูลในรูปแบบตารางที่มีคอลัมน์และแถว

เป็นรูปแบบที่ได้รับความนิยมสูงสุดตั้งแต่ทศวรรษ 1980

เนื่องจากมีประสิทธิภาพและความยืดหยุ่นสูงในการเข้าถึงข้อมูลที่มีโครงสร้าง

9.2 ฐานข้อมูลเชิงวัตถุ (Object-oriented Database)

จัดเก็บข้อมูลในรูปแบบอ็อบเจกต์ (Object) คล้ายกับแนวคิดการเขียนโปรแกรมเชิงวัตถุ

9.3 ฐานข้อมูลแบบกระจาย (Distributed Database)

ประกอบด้วยไฟล์ข้อมูลตั้งแต่ 2 แห่งขึ้นไปที่กระจายอยู่ในสถานที่ต่าง ๆ

อาจอยู่ในเครื่องคอมพิวเตอร์หลายเครื่องในสถานที่เดียวกัน

หรือกระจายอยู่ในเครือข่ายที่แตกต่างกันก็ได้

9.4 ฐานข้อมูลแบบ NoSQL (ไม่ใช่เชิงสัมพันธ์)

ฐานข้อมูล NoSQL ไม่มีโครงสร้าง Schema ที่ตายตัวเหมือนฐานข้อมูลเชิงสัมพันธ์

ถูกออกแบบมาเพื่อรองรับความยืดหยุ่นและการขยายตัวของข้อมูลที่ไม่มีโครงสร้าง เช่น ข้อความ วิดีโอ และรูปภาพ โดยแบ่งออกเป็นประเภทย่อยได้ดังนี้

- **Key-value store:** จัดเก็บข้อมูลเป็นคู่ key-value

นิยมใช้สำหรับแคชข้อมูลหรือข้อมูลตารางสินค้า ตัวอย่างเช่น Redis และ Memcached

- **Document store:** จัดเก็บข้อมูลในรูปแบบเอกสาร มักอยู่ในรูปแบบ JSON, XML หรือ BSON เหมาะกับข้อมูลกึ่งโครงสร้าง ตัวอย่างเช่น MongoDB

- **Wide-column store:** จัดเก็บข้อมูลเป็นคอลัมน์

ทำให้เข้าถึงเฉพาะคอลัมน์ที่ต้องการได้โดยไม่สิ้นเปลืองหน่วยความจำ ตัวอย่างเช่น Apache HBase และ Apache Cassandra

- **Graph store:** จัดเก็บข้อมูลในรูปแบบโหนด (Node) เส้นเชื่อม (Edge) และคุณสมบัติ (Property) เหมาะสำหรับข้อมูลที่เป็นเครือข่ายความสัมพันธ์ ตัวอย่างเช่น Neo4j

ทั้งนี้ ฐานข้อมูลบางประเภทให้ความสำคัญกับความพร้อมใช้งาน (Availability)

มากกว่าความสอดคล้องของข้อมูล (Consistency) ตามแนวคิด CAP Theorem (Consistency, Availability, Partition Tolerance)

ในขณะที่ฐานข้อมูลเชิงสัมพันธ์มักให้ความสำคัญกับความสอดคล้องของข้อมูลเป็นหลัก

10. ข้อดีของฐานข้อมูลเชิงสัมพันธ์

- ลดความซ้ำซ้อนของข้อมูล (Data Redundancy) ผ่านกระบวนการ Normalization
- รองรับผู้ใช้งานหลายคนพร้อมกัน (Multi-user access) พร้อมระบบควบคุมสิทธิ์การเข้าถึง
- มีความเป็นธุรกรรม (Transactional) รับประกันความสอดคล้องของข้อมูลตลอดเวลา
- รองรับการสำรองและกู้คืนข้อมูลได้ง่าย แม้ในขณะที่ฐานข้อมูลกำลังทำงานอยู่
- มีชุมชนผู้ใช้งานขนาดใหญ่ เนื่องจากมีการใช้งานมาอย่างยาวนาน
- Stored Procedure ช่วยลดงานที่ต้องทำซ้ำ และช่วยบริหารจัดการสิทธิ์การเข้าถึงข้อมูล

- การใช้ดัชนี (Index) ช่วยให้ค้นหาข้อมูลได้รวดเร็ว
โดยไม่ต้องตรวจสอบข้อมูลทุกแถวในตาราง

11. ความแตกต่างระหว่างฐานข้อมูลกับสเปรดชีต

ทั้งฐานข้อมูลและสเปรดชีต (เช่น Microsoft Excel)

ต่างก็เป็นเครื่องมือที่สะดวกสำหรับจัดเก็บข้อมูล แต่มีความแตกต่างกันในประเด็นสำคัญ ดังนี้

ประเด็น	สเปรดชีต (เช่น Excel)	ฐานข้อมูล (Database)
การจัดเก็บและจัดการข้อมูล	เหมาะกับผู้ใช้คนเดียวหรือกลุ่มเล็ก ไม่เหมาะกับข้อมูลที่ซับซ้อนมาก	ออกแบบให้จัดเก็บข้อมูลจำนวนมากอย่างเป็น รองรับการค้นหาที่ซับซ้อน
การเข้าถึงข้อมูล	เหมาะกับผู้ใช้จำนวนน้อย	รองรับผู้ใช้จำนวนมากเข้าถึงพร้อมกันได้โดย ผ่านภาษา SQL
ปริมาณข้อมูลที่รองรับ	มีข้อจำกัดด้านปริมาณข้อมูล	รองรับข้อมูลปริมาณมหาศาลได้

12. ความท้าทายของฐานข้อมูลในปัจจุบัน

- ปริมาณข้อมูลที่เพิ่มขึ้นอย่างรวดเร็วจากเซนเซอร์ เครื่องจักรที่เชื่อมต่อกัน
และแหล่งข้อมูลอื่น ๆ ทำให้ผู้ดูแลระบบต้องบริหารจัดการข้อมูลอย่างมีประสิทธิภาพ
- ภัยคุกคามด้านความปลอดภัยของข้อมูล (Data Breach) ที่เพิ่มมากขึ้น
ทำให้ต้องรักษาความปลอดภัยควบคู่ไปกับการเข้าถึงข้อมูลที่สะดวก
- ความต้องการเข้าถึงข้อมูลแบบเรียลไทม์ (Real-time)
เพื่อสนับสนุนการตัดสินใจทางธุรกิจที่รวดเร็ว

13. แนวโน้มในอนาคต: ฐานข้อมูลอัตโนมัติ (Autonomous Database)

ฐานข้อมูลอัตโนมัติใช้เทคโนโลยีคลาวด์และ Machine Learning

เพื่อทำงานประจำวันโดยอัตโนมัติ เช่น การปรับแต่งประสิทธิภาพ (Tuning)

การรักษาความปลอดภัย การสำรองข้อมูล และการอัปเดตระบบ

ช่วยลดภาระงานที่ต้องทำด้วยมือของผู้ดูแลระบบ (Database Administrator)

และช่วยเพิ่มประสิทธิภาพ ลดต้นทุน รวมถึงเพิ่มความปลอดภัยของข้อมูลในระยะยาว

RDBMS คืออะไร

RDBMS คือระบบจัดการฐานข้อมูลที่จัดเก็บข้อมูลในรูปแบบ 'ตาราง' (Table)

ซึ่งประกอบด้วยแถว (Row) และคอลัมน์ (Column) โดยแต่ละแถวคือระเบียบข้อมูลหนึ่งรายการ

และแต่ละคอลัมน์คือฟิลด์ข้อมูลหนึ่งประเภท RDBMS

เป็นพื้นฐานของระบบฐานข้อมูลสมัยใหม่แทบทั้งหมด

ตัวอย่างตาราง Customers:

CustomerID	CustomerName	City	Country
1	Alfreds Futterkiste	Berlin	Germany
2	Ana Trujillo	México D.F.	Mexico
3	Antonio Moreno	México D.F.	Mexico

หมายเหตุ: SQL เป็นมาตรฐาน ANSI/ISO

แต่ระบบฐานข้อมูลแต่ละยี่ห้ออาจมีส่วนขยายหรือรายละเอียดปลีกย่อยที่แตกต่างกันไปบ้าง

อย่างไรก็ตามคำสั่งหลัก เช่น *SELECT*, *UPDATE*, *DELETE*, *INSERT*

ยังคงทำงานในลักษณะใกล้เคียงกันในทุกระบบ

2. รูปแบบคำสั่ง SQL เบื้องต้น

การกระทำส่วนใหญ่ที่ต้องทำกับฐานข้อมูลจะอยู่ในรูปของ 'คำสั่ง SQL' (SQL Statement) ซึ่งประกอบด้วยคำสงวน (Keyword) ที่เข้าใจง่าย เช่น คำสั่งต่อไปนี้จะเลือกข้อมูลทั้งหมดจากตาราง Customers:

```
SELECT * FROM Customers;
```

ข้อควรทราบ

- คำสงวนของ SQL ไม่คำนึงถึงตัวพิมพ์เล็ก-ใหญ่ (ไม่ case sensitive) เช่น SELECT กับ select ถือว่าเหมือนกัน แต่ในเอกสารนี้จะเขียนคำสงวนด้วยตัวพิมพ์ใหญ่เพื่อความชัดเจน
- ระบบฐานข้อมูลบางระบบกำหนดให้ต้องใส่เครื่องหมายเซมิโคลอน (;) ปิดท้ายทุกคำสั่ง ซึ่งเป็นแนวปฏิบัติที่ดีและแนะนำให้ใส่เสมอเมื่อมีหลายคำสั่งในสคริปต์เดียวกัน

คำสั่ง SQL ที่สำคัญ

คำสั่ง	หน้าที่
SELECT	ดึงข้อมูลจากฐานข้อมูล
UPDATE	แก้ไขข้อมูลในตาราง
DELETE	ลบข้อมูลออกจากตาราง
INSERT INTO	เพิ่มข้อมูลใหม่เข้าตาราง
CREATE TABLE	สร้างตารางใหม่
DROP TABLE	ลบตารางทิ้งตาราง

3. คำสั่ง SELECT

คำสั่ง SELECT ใช้สำหรับเลือกข้อมูลจากฐานข้อมูล ผลลัพธ์ที่ได้จะถูกเก็บไว้ในรูปแบบตารางผลลัพธ์ (Result Set)

รูปแบบคำสั่ง (Syntax)

```
SELECT column1, column2, ...
FROM table_name;
```

โดย column1, column2 คือชื่อคอลัมน์ที่ต้องการเลือกจากราง และ table_name คือชื่อตารางที่ต้องการดึงข้อมูล

เลือกทุกคอลัมน์

หากต้องการเลือกทุกคอลัมน์ในตาราง สามารถใช้เครื่องหมายดอกจัน (*) แทนการระบุชื่อคอลัมน์ทีละคอลัมน์:

```
SELECT * FROM Customers;
```

ตัวอย่างการเลือกบางคอลัมน์

```
SELECT CustomerName, City FROM Customers;
```

4. คำสั่ง SELECT DISTINCT

คำสั่ง SELECT DISTINCT ใช้เพื่อคืนค่าเฉพาะแถวที่มีค่าไม่ซ้ำกัน (Unique Values) เท่านั้น ภายในตารางหนึ่งอาจมีค่าซ้ำกันในหลายแถว ซึ่ง DISTINCT จะช่วยตัดความซ้ำซ้อนของผลลัพธ์ออก

รูปแบบคำสั่ง

```
SELECT DISTINCT column1, column2, ...
FROM table_name;
```

ตัวอย่าง

คำสั่งต่อไปนี้จะคืนค่าเฉพาะประเทศที่ไม่ซ้ำกันจากราง Customers:

```
SELECT DISTINCT Country FROM Customers;
```

นับจำนวนค่าที่ไม่ซ้ำกัน

มาตรฐาน SQL ไม่มีฟังก์ชันนับค่าที่ไม่ซ้ำกันโดยตรงในบางระบบ (เช่น MS Access) จึงมักใช้ COUNT ร่วมกับ DISTINCT เพื่อหาจำนวนค่าที่ไม่ซ้ำกัน:

```
SELECT COUNT(DISTINCT Country) FROM Customers;
```

5. อนุประโยค WHERE

อนุประโยค WHERE ใช้สำหรับกรองระเบียบข้อมูลให้เหลือเฉพาะแถวที่ตรงตามเงื่อนไขที่กำหนด

รูปแบบคำสั่ง

```
SELECT column1, column2, ...  
FROM table_name  
WHERE condition;
```

ข้อความ (Text) กับ ตัวเลข (Numeric)

ค่าที่เป็นข้อความในเงื่อนไขต้องอยู่ภายในเครื่องหมายคำพูดเดี่ยว (single quotes)

ในขณะที่ค่าตัวเลขไม่จำเป็นต้องใส่เครื่องหมายคำพูด:

```
SELECT * FROM Customers WHERE Country = 'Mexico';
```

```
SELECT * FROM Customers WHERE CustomerID = 1;
```

ตัวดำเนินการที่ใช้ใน WHERE

ตัวดำเนินการ	ความหมาย
=	เท่ากับ
<> หรือ !=	ไม่เท่ากับ
>	มากกว่า
<	น้อยกว่า
>=	มากกว่าหรือเท่ากับ
<=	น้อยกว่าหรือเท่ากับ
BETWEEN	อยู่ในช่วงที่กำหนด
LIKE	ค้นหาตามรูปแบบข้อความ
IN	ระบุค่าที่เป็นไปได้หลายค่า

6. คำสั่ง ORDER BY

คำสั่ง ORDER BY ใช้สำหรับเรียงลำดับผลลัพธ์ที่ได้จากคำสั่ง SELECT โดยค่าเริ่มต้นจะเรียงจากน้อยไปมาก (ascending)

รูปแบบคำสั่ง

```
SELECT column1, column2, ...
FROM table_name
ORDER BY column1, column2, ... ASC|DESC;
```

เรียงจากมากไปน้อย

ใช้คำสั่ง DESC เพื่อเรียงลำดับจากมากไปน้อย:


```
SELECT * FROM Products
ORDER BY Price DESC;
```

เรียงลำดับตามตัวอักษร

```
SELECT * FROM Products
ORDER BY ProductName;
```

เรียงลำดับหลายคอลัมน์

สามารถระบุ ASC หรือ DESC แยกกันในแต่ละคอลัมน์ได้:

```
SELECT * FROM Customers
ORDER BY Country ASC, CustomerName DESC;
```

7. ตัวดำเนินการ AND

ตัวดำเนินการ AND ใช้ในอนุประโยค WHERE เพื่อกรองข้อมูลโดยต้องเป็นจริง 'ทุกเงื่อนไข' ที่กำหนดพร้อมกัน จึงจะถูกรวมอยู่ในผลลัพธ์

รูปแบบคำสั่ง

```
SELECT column1, column2, ...
FROM table_name
WHERE condition1 AND condition2 AND condition3 ...;
```

ตัวอย่าง

คำสั่งต่อไปนี้เลือกลูกค้าทั้งหมดจากประเทศสเปนที่ชื่อขึ้นต้นด้วยตัวอักษร 'G':

```
SELECT * FROM Customers
```

```
WHERE Country = 'Spain' AND CustomerName LIKE 'G%';
```

หมายเหตุ: สามารถผสม AND กับ OR ร่วมกันได้ โดยควรใช้วงเล็บ ()
เพื่อกำหนดลำดับความสำคัญของเงื่อนไขให้ชัดเจน

8. ตัวดำเนินการ OR

ตัวดำเนินการ OR ใช้กรองข้อมูลโดยขอเพียง 'อย่างน้อยหนึ่งเงื่อนไข' เป็นจริงเท่านั้น
ก็จะถูกรวมอยู่ในผลลัพธ์

รูปแบบคำสั่ง

```
SELECT column1, column2, ...  
FROM table_name  
WHERE condition1 OR condition2 OR condition3 ...;
```

ตัวอย่าง

คำสั่งต่อไปนี้เลือกลูกค้าทั้งหมดที่มาจากประเทศเยอรมนี หรือ สเปน:

```
SELECT * FROM Customers  
WHERE Country = 'Germany' OR Country = 'Spain';
```

การผสม AND และ OR

```
SELECT * FROM Customers  
WHERE Country = 'Germany'  
AND (City = 'Berlin' OR City = 'München');
```

9. ตัวดำเนินการ NOT

ตัวดำเนินการ NOT ใช้เพื่อกลับค่าความจริงของเงื่อนไข (negate)
หรือใช้ร่วมกับตัวดำเนินการอื่นเพื่อคัดข้อมูลที่ 'ไม่ตรง' ตามเงื่อนไขที่กำหนดออกจากผลลัพธ์

รูปแบบการใช้งานที่พบบ่อย

รูปแบบ	ความหมาย
NOT LIKE	คัดข้อมูลที่ไม่ตรงกับรูปแบบข้อความที่กำหนด
NOT BETWEEN	คัดข้อมูลที่อยู่นอกช่วงค่าที่กำหนด
NOT IN	คัดข้อมูลที่ไม่อยู่ในรายการค่าที่กำหนด
NOT >, NOT <	กลับความหมายของเงื่อนไขมากกว่า/น้อยกว่า

ตัวอย่าง

เลือกลูกค้าทั้งหมดที่ชื่อไม่ได้ขึ้นต้นด้วยตัวอักษร 'A':

```
SELECT * FROM Customers
WHERE NOT CustomerName LIKE 'A%';
```

เลือกลูกค้าที่เมืองไม่ใช่ 'Paris' และไม่ใช่ 'London':

```
SELECT * FROM Customers
WHERE City NOT IN ('Paris', 'London');
```

10. คำสั่ง INSERT INTO

คำสั่ง INSERT INTO ใช้สำหรับเพิ่มระเบียบข้อมูลใหม่เข้าไปในตาราง
สามารถเขียนได้สองรูปแบบหลัก

รูปแบบที่ 1: ระบุชื่อคอลัมน์และค่าที่ต้องการเพิ่ม

```
INSERT INTO table_name (column1, column2, column3, ...)
```

```
VALUES (value1, value2, value3, ...);
```

รูปแบบที่ 2: เพิ่มค่าครบทุกคอลัมน์ (ไม่ต้องระบุชื่อคอลัมน์)

หากใส่ค่าครบทุกคอลัมน์ของตาราง สามารถละชื่อคอลัมน์ได้
แต่ต้องเรียงลำดับค่าให้ตรงกับลำดับคอลัมน์ในตาราง:

```
INSERT INTO table_name VALUES (value1, value2, value3, ...);
```

ตัวอย่าง

```
INSERT INTO Customers (CustomerName, City, Country)
VALUES ('Cardinal', 'Stavanger', 'Norway');
```

เพิ่มข้อมูลหลายแถวพร้อมกัน

สามารถเพิ่มหลายระเบียนได้ในคำสั่งเดียว โดยคั่นแต่ละชุดค่าด้วยเครื่องหมายจุลภาค:

```
INSERT INTO Customers (CustomerName, City, Country)
VALUES
('Cardinal', 'Stavanger', 'Norway'),
('Greasy Burger', 'Feltre', 'Italy'),
('Tasty Tee', 'Bologna', 'Italy');
```

11. ค่า NULL

ค่า NULL หมายถึงฟิลด์ที่ไม่มีค่าอยู่เลย เป็นการแสดงถึงข้อมูลที่ไม่ทราบ ไม่มี
หรือไม่สามารถใช้ได้ ไม่ใช่ค่าว่างเปล่า (empty string) หรือค่าศูนย์ NULL เป็นเพียงตัวบ่งชี้ว่า
'ไม่มีข้อมูล' ในฟิลด์นั้น

การทดสอบค่า NULL

ไม่สามารถใช้ตัวดำเนินการเปรียบเทียบทั่วไป เช่น = หรือ <> กับค่า NULL ได้
จำเป็นต้องใช้ตัวดำเนินการ IS NULL และ IS NOT NULL แทน

รูปแบบคำสั่ง

```
SELECT column_names
FROM table_name
WHERE column_name IS NULL;
```

```
SELECT column_names
FROM table_name
WHERE column_name IS NOT NULL;
```

ตัวอย่าง

เลือกลูกค้าทั้งหมดที่ไม่มีข้อมูลในฟิลด์ Address:

```
SELECT CustomerName, Address
FROM Customers
WHERE Address IS NULL;
```

12. คำสั่ง UPDATE

คำสั่ง UPDATE ใช้สำหรับแก้ไขข้อมูลที่มีอยู่แล้วในตาราง

รูปแบบคำสั่ง

```
UPDATE table_name
SET column1 = value1, column2 = value2, ...
WHERE condition;
```

ตัวอย่าง

```
UPDATE Customers
SET ContactName = 'Alfred Schmidt', City = 'Frankfurt'
WHERE CustomerID = 1;
```

แก้ไขหลายระเบียนพร้อมกัน

```
UPDATE Customers
SET ContactName = 'Juan'
WHERE Country = 'Mexico';
```

หมายเหตุ: หากไม่ระบุประโยค *WHERE* คำสั่ง *UPDATE* จะแก้ไขข้อมูล 'ทุกแถว' ในตาราง ควรตรวจสอบเงื่อนไขให้ถูกต้องทุกครั้งก่อนรันคำสั่งจริง

13. คำสั่ง DELETE

คำสั่ง *DELETE* ใช้สำหรับลบระเบียนข้อมูลที่มีอยู่แล้วออกจากตาราง

รูปแบบคำสั่ง

```
DELETE FROM table_name WHERE condition;
```

ตัวอย่าง

```
DELETE FROM Customers WHERE CustomerName = 'Alfreds Futterkiste';
```

ลบข้อมูลทั้งหมดในตาราง

สามารถลบทุกแถวในตารางได้โดยไม่ระบุ *WHERE* ซึ่งจะเหลือตารางเปล่าไว้ (โครงสร้างตารางยังคงอยู่):

```
DELETE FROM table_name;
```

ลบตารางทั้งตาราง

หากต้องการลบทั้งตารางและโครงสร้าง ให้ใช้คำสั่ง DROP TABLE แทน:

```
DROP TABLE table_name;
```

14. การจำกัดจำนวนผลลัพธ์ (SELECT TOP / LIMIT)

ในบางกรณีต้องการจำกัดจำนวนระเบียบนที่คืนกลับมา

ไวยากรณ์ที่ใช้จะแตกต่างกันไปตามระบบฐานข้อมูล

SQL Server / MS Access — ใช้ TOP

```
SELECT TOP 3 * FROM Customers;
```

```
SELECT TOP 50 PERCENT * FROM Customers;
```

MySQL / PostgreSQL — ใช้ LIMIT

```
SELECT * FROM Customers LIMIT 3;
```

Oracle — ใช้ FETCH FIRST

```
SELECT * FROM Customers FETCH FIRST 3 ROWS ONLY;
```

ใช้ร่วมกับ WHERE และ ORDER BY

สามารถระบุเงื่อนไขและการเรียงลำดับร่วมกับการจำกัดจำนวนผลลัพธ์ได้ เช่น เลือก 3 อันดับแรกที่เรียงตามชื่อลูกค้า (ตัวอย่างสำหรับ MySQL):

```
SELECT * FROM Customers
ORDER BY CustomerName
LIMIT 3;
```

15. ฟังก์ชันรวมข้อมูล (Aggregate Functions)

ฟังก์ชันรวมข้อมูลใช้สำหรับคำนวณค่าสรุปจากกลุ่มของแถวข้อมูล เช่น หาค่าน้อยที่สุด มากที่สุด นับจำนวน หรือรวมผลรวม ฟังก์ชันเหล่านี้มักใช้ร่วมกับอนุประโยค GROUP BY เพื่อสรุปผลแยกตามกลุ่มข้อมูล

ฟังก์ชัน	หน้าที่
MIN()	คืนค่าน้อยที่สุดของคอลัมน์ที่เลือก
MAX()	คืนค่ามากที่สุดของคอลัมน์ที่เลือก
COUNT()	นับจำนวนแถวที่ตรงตามเงื่อนไข
SUM()	รวมค่าตัวเลขทั้งหมดในคอลัมน์
AVG()	หาค่าเฉลี่ยของคอลัมน์ตัวเลข

หมายเหตุ: ฟังก์ชันรวมข้อมูล (ยกเว้น COUNT(*)) จะไม่นำค่า NULL มาคำนวณด้วย

16. ฟังก์ชัน MIN()

ฟังก์ชัน MIN() คืนค่าที่น้อยที่สุดของคอลัมน์ที่เลือก สามารถใช้ได้กับข้อมูลประเภทตัวเลข ข้อความ และวันที่

รูปแบบคำสั่ง

```
SELECT MIN(column_name)
FROM table_name
WHERE condition;
```


ตัวอย่าง

```
SELECT MIN(Price) FROM Products;
```

ตั้งชื่อคอลัมน์ผลลัพธ์ (Alias)

```
SELECT MIN(Price) AS SmallestPrice  
FROM Products;
```

ใช้กับคอลัมน์วันที่

```
SELECT MIN(BirthDate) AS EarliestBirthdate  
FROM Employees;
```

ใช้ร่วมกับ GROUP BY

หาราคาต่ำสุดของสินค้าแยกตามแต่ละหมวดหมู่:

```
SELECT MIN(Price) AS SmallestPrice, CategoryID  
FROM Products  
GROUP BY CategoryID;
```

17. ฟังก์ชัน MAX()

ฟังก์ชัน MAX() คำนวณค่าที่มากที่สุดของคอลัมน์ที่เลือก ใช้ได้กับข้อมูลประเภทตัวเลข ข้อความ และวันที่ เช่นเดียวกับ MIN()

รูปแบบคำสั่ง

```
SELECT MAX(column_name)  
FROM table_name
```

```
WHERE condition;
```

ตัวอย่าง

```
SELECT MAX(Price) FROM Products;
```

ตั้งชื่อคอลัมน์ผลลัพธ์ (Alias)

```
SELECT MAX(Price) AS HighestPrice  
FROM Products;
```

ใช้กับคอลัมน์วันที่

```
SELECT MAX(BirthDate) AS LatestBirthdate  
FROM Employees;
```

ใช้ร่วมกับ GROUP BY

```
SELECT MAX(Price) AS HighestPrice, CategoryID  
FROM Products  
GROUP BY CategoryID;
```

18. ฟังก์ชัน COUNT()

ฟังก์ชัน COUNT() ใช้นับจำนวนแถวที่ตรงตามเงื่อนไขที่กำหนด
พฤติกรรมของฟังก์ชันจะขึ้นอยู่กับอาร์กิวเมนต์ที่ใส่ในวงเล็บ

รูปแบบคำสั่ง

```
SELECT COUNT(column_name)
```

```
FROM table_name
WHERE condition;
```

COUNT(*) — นับทุกแถว

นับจำนวนแถวทั้งหมดในตาราง รวมถึงแถวที่มีค่า NULL ด้วย:

```
SELECT COUNT(*) FROM Products;
```

COUNT(column_name) — นับเฉพาะค่าที่ไม่ใช่ NULL

```
SELECT COUNT(ProductName) FROM Products;
```

COUNT(DISTINCT column_name) — นับค่าที่ไม่ซ้ำกัน

นับเฉพาะค่าที่ไม่ซ้ำกันและไม่เป็น NULL ในคอลัมน์:

```
SELECT COUNT(DISTINCT Price) FROM Products;
```

ใช้ร่วมกับ WHERE และ GROUP BY

```
SELECT COUNT(*) AS TotalProducts
FROM Products
WHERE Price > 20;
```

19. ฟังก์ชัน SUM()

ฟังก์ชัน SUM() ใช้คำนวณผลรวมของค่าตัวเลขทั้งหมดในคอลัมน์ที่ระบุ โดยจะไม่นำค่า NULL มารวมด้วย

รูปแบบคำสั่ง

```
SELECT SUM(column_name)
FROM table_name
WHERE condition;
```

ตัวอย่าง

หาผลรวมจำนวนสินค้าที่สั่งซื้อทั้งหมดจากตาราง OrderDetails:

```
SELECT SUM(Quantity) FROM OrderDetails;
```

ใช้ร่วมกับ WHERE

```
SELECT SUM(Quantity)
FROM OrderDetails
WHERE ProductId = 11;
```

ตั้งชื่อคอลัมน์ผลลัพธ์ (Alias)

```
SELECT SUM(Quantity) AS TotalQuantity
FROM OrderDetails;
```

ใช้ร่วมกับ GROUP BY

```
SELECT OrderID, SUM(Quantity) AS [Total Quantity]
FROM OrderDetails
GROUP BY OrderID;
```

การใช้นิพจน์ภายใน SUM()

พารามิเตอร์ภายในฟังก์ชัน SUM() สามารถเป็นนิพจน์ทางคณิตศาสตร์ได้ เช่น การคูณจำนวนสินค้ากับราคาต่อหน่วยเพื่อหายอดขายรวมมูลค่า:

```
SELECT SUM(Quantity * Price) AS TotalValue  
FROM OrderDetails  
JOIN Products ON OrderDetails.ProductID = Products.ProductID;
```

20. ฟังก์ชัน AVG()

ฟังก์ชัน AVG() ใช้คำนวณค่าเฉลี่ยของคอลัมน์ตัวเลข โดยจะไม่นำค่า NULL มารวมคำนวณด้วย

รูปแบบคำสั่ง

```
SELECT AVG(column_name)
FROM table_name
WHERE condition;
```

ตัวอย่าง

```
SELECT AVG(Price) FROM Products;
```

ใช้ร่วมกับ WHERE

```
SELECT AVG(Price)
FROM Products
WHERE CategoryID = 1;
```

ตั้งชื่อคอลัมน์ผลลัพธ์ (Alias)

```
SELECT AVG(Price) AS [average price]
FROM Products;
```

ค้นหาที่สูงกว่าค่าเฉลี่ย

สามารถใช้ AVG() ร่วมกับคำสั่งย่อย (Subquery) เพื่อหาแถวที่มีค่าสูงกว่าค่าเฉลี่ยทั้งหมดได้:

```
SELECT * FROM Products
WHERE Price > (SELECT AVG(Price) FROM Products);
```

ใช้ร่วมกับ GROUP BY

```
SELECT AVG(Price) AS AveragePrice, CategoryID
FROM Products
GROUP BY CategoryID;
```

21. ตัวดำเนินการ LIKE และไวลด์การ์ด

ตัวดำเนินการ LIKE ใช้ในอนุประโยค WHERE เพื่อค้นหาข้อมูลตามรูปแบบที่กำหนด (Pattern Matching) มักใช้งานร่วมกับสัญลักษณ์ไวลด์การ์ด (Wildcard) เพื่อแทนตัวอักษรที่ไม่ทราบแน่ชัด

รูปแบบคำสั่ง

```
SELECT column1, column2, ...
FROM table_name
WHERE columnN LIKE pattern;
```

สัญลักษณ์ไวลด์การ์ดหลัก

สัญลักษณ์	ความหมาย
%	แทนตัวอักษรจำนวนเท่าใดก็ได้ ตั้งแต่ศูนย์ตัวขึ้นไป
_	แทนตัวอักษรเพียง 1 ตัว

สัญลักษณ์	ความหมาย
[charlist]	แทนตัวอักษรตัวใดตัวหนึ่งในกลุ่มที่ระบุ (รองรับเฉพาะบางระบบ เช่น SQL Server/MS Access)
[!charlist] หรือ [^charlist]	แทนตัวอักษรที่ไม่อยู่ในกลุ่มที่ระบุ

ตัวอย่างการใช้งาน %

ค้นหาลูกค้าที่ชื่อขึ้นต้นด้วยตัวอักษร 'a':

```
SELECT * FROM Customers
WHERE CustomerName LIKE 'a%';
```

ค้นหาลูกค้าที่เมืองมีคำว่า 'on' ปรากฏอยู่ตรงไหนก็ได้:

```
SELECT * FROM Customers
WHERE City LIKE '%on%';
```

ค้นหาลูกค้าที่ชื่อลงท้ายด้วย 'es':

```
SELECT * FROM Customers
WHERE CustomerName LIKE '%es';
```

ตัวอย่างการใช้งาน _

ค้นหาลูกค้าที่เมืองขึ้นต้นด้วยตัวอักษรใดก็ได้ 1 ตัว ตามด้วย 'ondon':

```
SELECT * FROM Customers
WHERE City LIKE '_ondon';
```

ผสมไวด์การ์ดหลายแบบ

ค้นหาลูกค้าที่ชื่อขึ้นต้นด้วย 'a' และมีความยาวอย่างน้อย 3 ตัวอักษร:

```
SELECT * FROM Customers
WHERE CustomerName LIKE 'a__%';
```

การใช้งานกลุ่มตัวอักษร [] (SQL Server / MS Access)

ค้นหาลูกค้าที่ชื่อขึ้นต้นด้วยตัวอักษร b, s หรือ p:

```
SELECT * FROM Customers
WHERE CustomerName LIKE '[bsp]%';
```

ค้นหาลูกค้าที่ชื่อขึ้นต้นด้วยตัวอักษรในช่วง a ถึง f:

```
SELECT * FROM Customers
WHERE CustomerName LIKE '[a-f]%';
```

หมายเหตุ: ถ้าไม่ใส่สัญลักษณ์ไวลด์การ์ดเลย ข้อความในเงื่อนไข LIKE จะต้องตรงกันแบบทั้งหมด (exact match) จึงจะได้ผลลัพธ์

22. ตัวดำเนินการ IN

ตัวดำเนินการ IN ใช้เป็นทางเลือกแทนการเขียนเงื่อนไข OR หลายค่าติดกัน ทำให้คำสั่งกระชับและอ่านง่ายขึ้น

รูปแบบคำสั่ง

```
SELECT column_name(s)
FROM table_name
WHERE column_name IN (value1, value2, ...);
```

ตัวอย่าง

เลือกลูกค้าที่มาจากประเทศเยอรมนี ฝรั่งเศส หรือสหราชอาณาจักร:

```
SELECT * FROM Customers
WHERE Country IN ('Germany', 'France', 'UK');
```

NOT IN

คัดลอกค่าที่ไม่ได้มาจากประเทศเหล่านั้นออก:

```
SELECT * FROM Customers
WHERE Country NOT IN ('Germany', 'France', 'UK');
```

IN ร่วมกับ Subquery

สามารถใช้ IN ร่วมกับคำสั่งย่อยเพื่อเลือกลูกค้าที่มีคำสั่งซื้ออยู่ในตาราง Orders:

```
SELECT * FROM Customers
WHERE CustomerID IN (SELECT CustomerID FROM Orders);
```

23. ตัวดำเนินการ BETWEEN

ตัวดำเนินการ BETWEEN ใช้ในอนุประโยค WHERE เพื่อเลือกค่าที่อยู่ภายในช่วงที่กำหนด โดยรวมค่าเริ่มต้นและค่าสิ้นสุดของช่วงด้วย (inclusive) ใช้ได้ทั้งกับตัวเลข ข้อความ และวันที่

รูปแบบคำสั่ง

```
SELECT column_name(s)
FROM table_name
WHERE column_name BETWEEN value1 AND value2;
```

ตัวอย่างกับตัวเลข

```
SELECT * FROM Products
```

```
WHERE Price BETWEEN 10 AND 20;
```

NOT BETWEEN

```
SELECT * FROM Products
WHERE Price NOT BETWEEN 10 AND 20;
```

BETWEEN ร่วมกับ IN

```
SELECT * FROM Products
WHERE Price BETWEEN 10 AND 20
AND CategoryID NOT IN (1, 2, 3);
```

BETWEEN กับข้อความ

```
SELECT * FROM Products
WHERE ProductName BETWEEN 'Carnarvon Tigers' AND 'Mozzarella di Giovanni'
ORDER BY ProductName;
```

BETWEEN กับวันที่

```
SELECT * FROM Orders
WHERE OrderDate BETWEEN '1996-07-01' AND '1996-07-31';
```

24. นามแฝง (Aliases)

นามแฝง (Alias) ใช้เพื่อตั้งชื่อชั่วคราวให้กับคอลัมน์หรือตาราง เพื่อให้ผลลัพธ์อ่านง่ายขึ้นหรือคำสั่งกระชับขึ้น โดยนามแฝงจะมีผลเฉพาะในคำสั่งนั้น ๆ เท่านั้น

นามแฝงสำหรับคอลัมน์

```
SELECT CustomerID AS ID, CustomerName AS Customer
FROM Customers;
```

นามแฝงที่มีช่องว่าง

หากต้องการใช้ชื่อที่มีช่องว่าง ให้ครอบด้วยเครื่องหมายวงเล็บเหลี่ยมหรือเครื่องหมายคำพูด:

```
SELECT ProductName AS [My Great Products]
FROM Products;
```

การต่อข้อความหลายคอลัมน์

```
SELECT CustomerName,
       Address + ', ' + PostalCode + ' ' + City + ', ' + Country AS Address
FROM Customers;
```

นามแฝงสำหรับตาราง

นามแฝงของตารางมีประโยชน์มากเมื่อคำสั่งเกี่ยวข้องกับหลายตาราง เพราะช่วยให้คำสั่งสั้นและอ่านง่ายขึ้น:

```
SELECT o.OrderID, o.OrderDate, c.CustomerName
FROM Customers AS c, Orders AS o
WHERE c.CustomerName = 'Around the Horn'
AND c.CustomerID = o.CustomerID;
```

25. อนุประโยค JOIN

อนุประโยค JOIN ใช้สำหรับรวมข้อมูลจากสองตารางหรือมากกว่าเข้าด้วยกัน โดยอาศัยคอลัมน์ที่มีความสัมพันธ์กันระหว่างตาราง

ประเภทของ JOIN ในภาษา SQL

ประเภท	ความหมาย
(INNER) JOIN	คืนเฉพาะแถวที่มีค่าตรงกันในทั้งสองตาราง
LEFT (OUTER) JOIN	คืนทุกแถวจากตารางซ้าย และแถวที่ตรงกันจากตารางขวา
RIGHT (OUTER) JOIN	คืนทุกแถวจากตารางขวา และแถวที่ตรงกันจากตารางซ้าย
FULL (OUTER) JOIN	คืนทุกแถวเมื่อมีค่าตรงกันในตารางใดตารางหนึ่ง

26. INNER JOIN

INNER JOIN คืนเฉพาะแถวที่มีค่าตรงกันในทั้งสองตารางเท่านั้น สามารถเขียนสั้น ๆ เป็น JOIN ได้เช่นกัน เนื่องจากเป็นชนิดการเชื่อมตารางเริ่มต้น

รูปแบบคำสั่ง

```
SELECT column_name(s)
FROM table1
INNER JOIN table2
ON table1.column_name = table2.column_name;
```

ตัวอย่าง

เชื่อมตาราง Products และ Categories ผ่านคอลัมน์ CategoryID ที่สัมพันธ์กัน:

```
SELECT ProductID, ProductName, CategoryName
FROM Products
INNER JOIN Categories ON Products.CategoryID = Categories.CategoryID;
```

เชื่อมหลายตารางพร้อมกัน

สามารถใส่อนุประโยค INNER JOIN หลายครั้งเพื่อเชื่อมมากกว่าสองตาราง เช่น รวมข้อมูลคำสั่งซื้อกับข้อมูลลูกค้าและผู้ขนส่ง:

```
SELECT Orders.OrderID, Customers.CustomerName, Shippers.ShipperName
FROM Orders
INNER JOIN Customers ON Orders.CustomerID = Customers.CustomerID
INNER JOIN Shippers ON Orders.ShipperID = Shippers.ShipperID;
```

27. LEFT JOIN

LEFT JOIN ค้นค่าทุกแถวจากตารางซ้าย (table1) และแถวที่มีค่าตรงกันจากตารางขวา (table2) หากไม่มีค่าตรงกันในตารางขวา ผลลัพธ์ของคอลัมน์จากตารางขวาจะเป็น NULL คำสงวน LEFT OUTER JOIN และ LEFT JOIN มีความหมายเดียวกัน

รูปแบบคำสั่ง

```
SELECT column_name(s)
FROM table1
LEFT JOIN table2
ON table1.column_name = table2.column_name;
```

ตัวอย่าง

แสดงลูกค้าทุกคนพร้อมคำสั่งซื้อ รวมถึงลูกค้าที่ยังไม่เคยสั่งซื้อสินค้าเลย:

```
SELECT Customers.CustomerName, Orders.OrderID
FROM Customers
LEFT JOIN Orders ON Customers.CustomerID = Orders.CustomerID
ORDER BY Customers.CustomerName;
```

หากต้องการหาเฉพาะลูกค้าที่ยังไม่เคยสั่งซื้อ ให้เพิ่มเงื่อนไข WHERE เพื่อกรองค่า NULL ในฝั่งตารางขวา:

```
SELECT Customers.CustomerName, Orders.OrderID
FROM Customers
LEFT JOIN Orders ON Customers.CustomerID = Orders.CustomerID
WHERE Orders.OrderID IS NULL;
```

28. RIGHT JOIN

RIGHT JOIN คืบค่าทุกแถวจากตารางขวา (table2) และแถวที่มีค่าตรงกันจากตารางซ้าย (table1) หากไม่มีค่าตรงกันในตารางซ้าย ผลลัพธ์ของคอลัมน์จากตารางซ้ายจะเป็น NULL คำสงวน RIGHT OUTER JOIN และ RIGHT JOIN มีความหมายเดียวกัน

รูปแบบคำสั่ง

```
SELECT column_name(s)
FROM table1
RIGHT JOIN table2
ON table1.column_name = table2.column_name;
```

ตัวอย่าง

แสดงพนักงานทุกคนพร้อมคำสั่งซื้อที่พวกเขาได้รับผิดชอบ รวมถึงพนักงานที่ยังไม่เคยรับคำสั่งซื้อใดเลย:

```
SELECT Orders.OrderID, Employees.LastName, Employees.FirstName
FROM Orders
RIGHT JOIN Employees ON Orders.EmployeeID = Employees.EmployeeID
ORDER BY Orders.OrderID;
```

29. FULL JOIN (FULL OUTER JOIN)

FULL JOIN คือนำค่าทุกแถวเมื่อมีความตรงกันในตารางใดตารางหนึ่ง (ตารางซ้ายหรือตารางขวา) หากแถวใดในตารางซ้ายไม่มีคู่ตรงกันในตารางขวา คอลัมน์จากตารางขวาจะเป็น NULL และในทางกลับกันก็เช่นเดียวกัน

รูปแบบคำสั่ง

```
SELECT column_name(s)
FROM table1
FULL OUTER JOIN table2
ON table1.column_name = table2.column_name
WHERE condition;
```

ตัวอย่าง

```
SELECT Customers.CustomerName, Orders.OrderID
FROM Customers
FULL OUTER JOIN Orders ON Customers.CustomerID = Orders.CustomerID
ORDER BY Customers.CustomerName;
```

หมายเหตุ: FULL JOIN อาจคืนผลลัพธ์ที่มีขนาดใหญ่มากได้
เนื่องจากรวมทุกแถวจากทั้งสองตารางเข้าด้วยกัน ควรใช้อย่างระมัดระวังกับตารางขนาดใหญ่

30. Self Join

Self Join คือการเชื่อมตารางกับตัวเอง โดยใช้หลักการเดียวกับ JOIN ทั่วไป แต่ตารางต้นทางและตารางปลายทางเป็นตารางเดียวกัน จึงจำเป็นต้องตั้งนามแฝง (Alias) ที่แตกต่างกันให้กับตารางทั้งสองฝั่งเพื่อแยกความแตกต่าง

รูปแบบคำสั่ง


```
SELECT column_name(s)
FROM table1 T1, table1 T2
WHERE condition;
```

ตัวอย่าง

ค้นหาลูกค้าที่อยู่ในเมืองเดียวกัน โดยเชื่อมตาราง Customers กับตัวเอง:

```
SELECT A.CustomerName AS CustomerName1,
       B.CustomerName AS CustomerName2, A.City
FROM Customers A, Customers B
WHERE A.CustomerID <> B.CustomerID
AND A.City = B.City
ORDER BY A.City;
```

31. ตัวดำเนินการ UNION

ตัวดำเนินการ UNION ใช้รวมผลลัพธ์จากคำสั่ง SELECT ตั้งแต่สองคำสั่งขึ้นไปเข้าด้วยกัน โดยจะคัดค่าที่ซ้ำกันออกโดยอัตโนมัติ (คืนเฉพาะค่าที่ไม่ซ้ำกัน)

เงื่อนไขการใช้งาน UNION

- แต่ละคำสั่ง SELECT ที่นำมารวมกันต้องมีจำนวนคอลัมน์เท่ากัน
- คอลัมน์ต้องมีชนิดข้อมูลที่ใกล้เคียงหรือเข้ากันได้
- คอลัมน์ในแต่ละคำสั่ง SELECT ต้องเรียงลำดับตรงกัน

รูปแบบคำสั่ง

```
SELECT column_name(s) FROM table1
UNION
```

```
SELECT column_name(s) FROM table2;
```

ตัวอย่าง

รวมรายชื่อประเทศที่ไม่ซ้ำกันจากตาราง Customers และ Suppliers:

```
SELECT Country FROM Customers
UNION
SELECT Country FROM Suppliers
ORDER BY Country;
```

UNION ร่วมกับ WHERE

```
SELECT City, Country FROM Customers
WHERE Country = 'Germany'
UNION
SELECT City, Country FROM Suppliers
WHERE Country = 'Germany'
ORDER BY City;
```

32. ตัวดำเนินการ UNION ALL

UNION ALL ทำงานคล้ายกับ UNION แต่จะคืนค่าทุกแถวรวมถึงค่าที่ซ้ำกันด้วย โดยไม่มีการตัดข้อมูลซ้ำออก ทำให้ประมวลผลได้เร็วกว่า UNION ในกรณีที่ไม่จำเป็นต้องคัดค่าซ้ำ

รูปแบบคำสั่ง

```
SELECT column_name(s) FROM table1
UNION ALL
SELECT column_name(s) FROM table2;
```

ตัวอย่าง

```
SELECT Country FROM Customers
UNION ALL
SELECT Country FROM Suppliers
ORDER BY Country;
```

UNION ALL ร่วมกับ WHERE

```
SELECT City, Country FROM Customers
WHERE Country = 'Germany'
UNION ALL
SELECT City, Country FROM Suppliers
WHERE Country = 'Germany'
ORDER BY City;
```

33. คำสั่ง GROUP BY

คำสั่ง GROUP BY ใช้จัดกลุ่มแถวที่มีค่าเหมือนกันในคอลัมน์ที่ระบุ ให้กลายเป็นแถวสรุปเดียว มักใช้ร่วมกับฟังก์ชันรวมข้อมูล เช่น COUNT(), SUM(), AVG() เพื่อสรุปผลแยกตามกลุ่ม

รูปแบบคำสั่ง

```
SELECT column_name(s)
FROM table_name
WHERE condition
GROUP BY column_name(s)
ORDER BY column_name(s);
```

ตัวอย่าง

นับจำนวนลูกค้าในแต่ละประเทศ:

```
SELECT COUNT(CustomerID), Country
FROM Customers
GROUP BY Country;
```

จัดเรียงผลลัพธ์จากมากไปน้อย:

```
SELECT COUNT(CustomerID), Country
FROM Customers
GROUP BY Country
ORDER BY COUNT(CustomerID) DESC;
```

GROUP BY ร่วมกับ JOIN

นับจำนวนคำสั่งซื้อที่จัดส่งโดยผู้ให้บริการขนส่งแต่ละราย:

```
SELECT Shippers.ShipperName, COUNT(Orders.OrderID) AS NumberOfOrders
FROM Orders
LEFT JOIN Shippers ON Orders.ShipperID = Shippers.ShipperID
GROUP BY ShipperName;
```

34. อนุประโยค HAVING

อนุประโยค HAVING ใช้กรองผลลัพธ์ที่ได้จากการจัดกลุ่มด้วย GROUP BY โดยอิงจากค่าที่ได้จากฟังก์ชันรวมข้อมูล ในขณะที่ WHERE ไม่สามารถใช้กรองผลลัพธ์ของฟังก์ชันรวมข้อมูลได้โดยตรง จึงต้องใช้ HAVING แทน

รูปแบบคำสั่ง

```
SELECT column_name(s)
```

```
FROM table_name
WHERE condition
GROUP BY column_name(s)
HAVING condition
ORDER BY column_name(s);
```

ตัวอย่าง

แสดงเฉพาะประเทศที่มีลูกค้ามากกว่า 5 ราย:

```
SELECT COUNT(CustomerID), Country
FROM Customers
GROUP BY Country
HAVING COUNT(CustomerID) > 5;
```

จัดเรียงผลลัพธ์จากมากไปน้อย พร้อมกรองเฉพาะประเทศที่มีลูกค้ามากกว่า 5 ราย:

```
SELECT COUNT(CustomerID), Country
FROM Customers
GROUP BY Country
HAVING COUNT(CustomerID) > 5
ORDER BY COUNT(CustomerID) DESC;
```

HAVING ร่วมกับ JOIN

แสดงเฉพาะพนักงานที่จัดการคำสั่งซื้อมากกว่า 10 รายการ:

```
SELECT Employees.LastName, COUNT(Orders.OrderID) AS NumberOfOrders
FROM Orders
INNER JOIN Employees ON Orders.EmployeeID = Employees.EmployeeID
```

```
GROUP BY LastName
HAVING COUNT(Orders.OrderID) > 10;
```

หมายเหตุ: ความแตกต่างสำคัญ: *WHERE* ใช้กรองแถวข้อมูลก่อนจัดกลุ่ม ส่วน *HAVING* ใช้กรองกลุ่มข้อมูลหลังจากคำนวณฟังก์ชันรวมแล้ว

35. ตัวดำเนินการ EXISTS

ตัวดำเนินการ EXISTS ใช้ตรวจสอบว่าคำสั่งย่อย (Subquery) คืนค่าอย่างน้อยหนึ่งแถวหรือไม่ หากมีอย่างน้อยหนึ่งแถว ผลลัพธ์จะเป็น TRUE มิฉะนั้นจะเป็น FALSE

รูปแบบคำสั่ง

```
SELECT column_name(s)
FROM table_name
WHERE EXISTS
(SELECT column_name FROM table_name WHERE condition);
```

ตัวอย่าง

เลือกซัพพลายเออร์ที่มีสินค้าราคาต่ำกว่า 20:

```
SELECT SupplierName
FROM Suppliers
WHERE EXISTS
(SELECT ProductName FROM Products
WHERE Products.SupplierID = Suppliers.supplierID AND Price < 20);
```

36. ตัวดำเนินการ ANY

ตัวดำเนินการ ANY ใช้ตรวจสอบเงื่อนไขกับค่าจากคำสั่งย่อย โดยจะเป็น TRUE หากมีอย่างน้อยหนึ่งค่าในผลลัพธ์ของคำสั่งย่อยที่ตรงตามเงื่อนไข

รูปแบบคำสั่ง

```
SELECT column_name(s)
FROM table_name
WHERE column_name operator ANY
(SELECT column_name FROM table_name WHERE condition);
```

ตัวอย่าง

เลือกสินค้าที่มีปริมาณสั่งซื้อเท่ากับ 10 อย่างน้อยหนึ่งรายการในตาราง OrderDetails:

```
SELECT ProductName
FROM Products
WHERE ProductID = ANY
(SELECT ProductID FROM OrderDetails WHERE Quantity = 10);
```

37. ตัวดำเนินการ ALL

ตัวดำเนินการ ALL ใช้ตรวจสอบเงื่อนไขกับค่าจากคำสั่งย่อย โดยจะเป็น TRUE ก็ต่อเมื่อ 'ทุกค่า' ในผลลัพธ์ของคำสั่งย่อยตรงตามเงื่อนไขที่กำหนด

รูปแบบคำสั่ง

```
SELECT column_name(s)
FROM table_name
WHERE column_name operator ALL
(SELECT column_name FROM table_name WHERE condition);
```

ตัวอย่าง

เลือกสินค้าที่มีปริมาณสั่งซื้อเท่ากับ 10 ในทุกรายการของตาราง OrderDetails:

```
SELECT ProductName
FROM Products
WHERE ProductID = ALL
(SELECT ProductID FROM OrderDetails WHERE Quantity = 10);
```

38. คำสั่ง SELECT INTO

คำสั่ง SELECT INTO ใช้คัดลอกข้อมูลจากตารางหนึ่งไปสร้างเป็นตารางใหม่ โดยตารางใหม่ที่ได้จะมีโครงสร้างคอลัมน์และชนิดข้อมูลตรงกับผลลัพธ์ของคำสั่ง SELECT นั้น

คัดลอกทั้งตาราง

```
SELECT * INTO NewTable FROM Customers;
```

คัดลอกเฉพาะบางคอลัมน์

```
SELECT CustomerName, ContactName
INTO NewTable
FROM Customers;
```

คัดลอกพร้อมเงื่อนไข

```
SELECT * INTO NewTable
FROM Customers
WHERE Country = 'Germany';
```

สร้างตารางสำรอง (Backup)

นิยมใช้ SELECT INTO เพื่อสำรองข้อมูลก่อนทำการแก้ไขหรือย้ายฐานข้อมูล:

```
SELECT * INTO CustomersBackup2024 FROM Customers;
```

คัดลอกข้อมูลจากหลายตาราง

สามารถใช้ร่วมกับ JOIN เพื่อรวมข้อมูลจากหลายตารางเข้าไปในตารางใหม่ได้:

```
SELECT Customers.CustomerName, Orders.OrderID  
INTO CustomersOrderBackup  
FROM Customers  
LEFT JOIN Orders ON Customers.CustomerID = Orders.CustomerID;
```

39. คำสั่ง INSERT INTO SELECT

คำสั่ง INSERT INTO SELECT

ใช้คัดลอกข้อมูลจากตารางหนึ่งไปเพิ่มเข้าในอีกตารางหนึ่งที่มีอยู่แล้ว
โดยข้อมูลเดิมที่มีอยู่ในตารางปลายทางจะไม่ได้รับผลกระทบใด ๆ

คัดลอกทุกคอลัมน์

```
INSERT INTO table2  
SELECT * FROM table1;
```

คัดลอกเฉพาะบางคอลัมน์

```
INSERT INTO table2 (column1, column2, column3, ...)  
SELECT column1, column2, column3, ...  
FROM table1  
WHERE condition;
```

ตัวอย่าง

คัดลอกข้อมูลซัพพลายเออร์ไปเป็นลูกค้าใหม่ในตาราง Customers:

```
INSERT INTO Customers (CustomerName, City, Country)  
SELECT SupplierName, City, Country  
FROM Suppliers;
```

คัดลอกพร้อมเงื่อนไข

```
INSERT INTO Customers (CustomerName, City, Country)
```

```
SELECT SupplierName, City, Country
FROM Suppliers
WHERE Country = 'Germany';
```

40. นิพจน์ CASE

นิพจน์ CASE ใช้สำหรับตรวจสอบเงื่อนไขแบบมีทางเลือกหลายทาง (คล้ายกับ if-else ในภาษาโปรแกรมทั่วไป) โดยจะไล่ตรวจสอบเงื่อนไขทีละข้อ เมื่อพบเงื่อนไขแรกที่เป็นจริงจะคืนค่าที่กำหนดไว้ทันที และหยุดตรวจสอบเงื่อนไขถัดไป หากไม่มีเงื่อนไขใดเป็นจริงเลย จะคืนค่าใน ELSE (ถ้ามี)

รูปแบบคำสั่ง

```
CASE
  WHEN condition1 THEN result1
  WHEN condition2 THEN result2
  WHEN conditionN THEN resultN
  ELSE result
END;
```

ตัวอย่าง

แสดงชื่อลูกค้าพร้อมข้อความอธิบายปริมาณสั่งซื้อในแต่ละคำสั่งซื้อ:

```
SELECT OrderID, Quantity,
CASE
  WHEN Quantity > 30 THEN 'The quantity is greater than 30'
  WHEN Quantity = 30 THEN 'The quantity is 30'
  ELSE 'The quantity is under 30'
```

```
END AS QuantityText
FROM OrderDetails;
```

ใช้ CASE ร่วมกับ ORDER BY เพื่อเรียงลำดับข้อมูลตามเงื่อนไขเฉพาะ:

```
SELECT CustomerName, City, Country
FROM Customers
ORDER BY
(CASE
  WHEN City IS NULL THEN Country
  ELSE City
END);
```

41. การจัดการค่า NULL: COALESCE(), IFNULL(), ISNULL(), NVL()

การคำนวณที่เกี่ยวข้องกับค่า NULL อาจให้ผลลัพธ์ที่ไม่คาดคิด เช่น หากนำค่าตัวเลขไปบวกกับ NULL ผลลัพธ์ที่ได้จะเป็น NULL เสมอ ภาษา SQL จึงมีฟังก์ชันสำหรับแทนที่ค่า NULL ด้วยค่าอื่นที่กำหนดไว้

COALESCE() — มาตรฐานที่แนะนำ

ฟังก์ชัน COALESCE() เป็นฟังก์ชันมาตรฐานที่ใช้ได้กับ MySQL, SQL Server และ Oracle ใช้แทนที่ค่า NULL ด้วยค่าที่กำหนด:

```
SELECT ProductName, (UnitPrice * (UnitsInStock + COALESCE(UnitsOnOrder, 0)))
FROM Products;
```

IFNULL() — สำหรับ MySQL

```
SELECT ProductName, (UnitPrice * (UnitsInStock + IFNULL(UnitsOnOrder, 0)))
```

```
FROM Products;
```

ISNULL() — สำหรับ SQL Server

```
SELECT ProductName, (UnitPrice * (UnitsInStock + ISNULL(UnitsOnOrder, 0)))
FROM Products;
```

NVL() — สำหรับ Oracle

```
SELECT ProductName, (UnitPrice * (UnitsInStock + NVL(UnitsOnOrder, 0)))
FROM Products;
```

หมายเหตุ: ฟังก์ชัน *ISNULL()* ในฐานข้อมูล *Microsoft Access* มีความหมายต่างออกไป คือใช้ตรวจสอบว่านิพจน์เป็น *NULL* หรือไม่ (คืนค่า *TRUE/FALSE*) ไม่ใช่การแทนที่ค่า *NULL*

42. โพรซีเยอร์ (Stored Procedure)

Stored Procedure คือชุดคำสั่ง SQL

ที่ถูกคอมไพล์ไว้ล่วงหน้าและสามารถบันทึกไว้เพื่อเรียกใช้งานซ้ำได้

ช่วยให้สามารถนำโค้ดเดิมกลับมาใช้ซ้ำได้จากหลายแอปพลิเคชัน

และช่วยเพิ่มความปลอดภัยและประสิทธิภาพในการทำงานกับฐานข้อมูล

รูปแบบคำสั่ง (SQL Server)

```
CREATE PROCEDURE procedure_name
AS
sql_statement
GO;
```

การเรียกใช้งาน Stored Procedure

```
EXEC procedure_name;
```

การลบ Stored Procedure

```
DROP PROCEDURE procedure_name;
```

ตัวอย่าง

```
CREATE PROCEDURE GetCustomersByCity @City nvarchar(30)
AS
SELECT * FROM Customers WHERE City = @City
GO;
```

```
EXEC GetCustomersByCity @City = 'London';
```

Stored Procedure ที่มีพารามิเตอร์หลายตัว

```
CREATE PROCEDURE GetCustomersByCity
    @City nvarchar(30), @PostalCode nvarchar(10)
AS
SELECT * FROM Customers
WHERE City = @City AND PostalCode = @PostalCode
GO;
```

```
EXEC GetCustomersByCity @City = 'London', @PostalCode = 'WA1 1DP';
```

43. คอมเมนต์ในภาษา SQL

คอมเมนต์ใช้เพื่ออธิบายโค้ด SQL หรือใช้ระงับการทำงานของบางส่วนของคำสั่งชั่วคราว (สำหรับการดีบั๊ก) โดยตัวประมวลผลฐานข้อมูลจะข้ามส่วนที่เป็นคอมเมนต์ไปทั้งหมด SQL รองรับทั้งคอมเมนต์บรรทัดเดียวและคอมเมนต์หลายบรรทัด

หมายเหตุ: ฐานข้อมูล Microsoft Access ไม่รองรับการใช้คอมเมนต์

คอมเมนต์บรรทัดเดียว (--)

ข้อความหลังเครื่องหมาย -- จนถึงท้ายบรรทัดจะถูกละเว้น:

```
-- เลือกลูกค้าทั้งหมดจากประเทศเม็กซิโก
SELECT * FROM Customers WHERE Country = 'Mexico';
```

ใช้เพื่อปิดการทำงานของบางส่วนของท้ายบรรทัด:

```
SELECT * FROM Customers
-- WHERE City='Berlin';
```

คอมเมนต์หลายบรรทัด (/* ... */)

```
/* เลือกทุกคอลัมน์จากลูกค้าที่
อยู่ในเมือง Berlin */
SELECT * FROM Customers WHERE City = 'Berlin';
```

ใช้เพื่อปิดการทำงานของคำสั่ง SQL หลายคำสั่งพร้อมกัน:

```
/*SELECT * FROM Customers;
SELECT * FROM Orders;*/
SELECT * FROM Suppliers;
```

ใช้เพื่อปิดการทำงานของเฉพาะบางส่วนของคำสั่ง:

```
SELECT CustomerName, /*City,*/ Country FROM Customers;
```

44. ตัวดำเนินการในภาษา SQL (SQL Operators)

ตัวดำเนินการ SQL คือคำสั่งและสัญลักษณ์ที่ใช้ในการกระทำต่าง ๆ กับค่าข้อมูล มักปรากฏอยู่ในอนุประโยค WHERE, HAVING และคำสั่งอื่น ๆ สามารถแบ่งออกเป็นหลายประเภทดังนี้

ตัวดำเนินการทางคณิตศาสตร์ (Arithmetic Operators)

ตัวดำเนินการ	ความหมาย
+	บวก
-	ลบ
*	คูณ
/	หาร
%	หารเอาเศษ

ตัวดำเนินการเปรียบเทียบ (Comparison Operators)

ตัวดำเนินการ	ความหมาย
=	เท่ากับ
>	มากกว่า
<	น้อยกว่า
>=	มากกว่าหรือเท่ากับ
<=	น้อยกว่าหรือเท่ากับ
<> หรือ !=	ไม่เท่ากับ

ตัวดำเนินการผสม (Compound Operators)

ตัวดำเนินการ	ความหมาย
+=	บวกแล้วกำหนดค่า
-=	ลบแล้วกำหนดค่า
*=	คูณแล้วกำหนดค่า
/=	หารแล้วกำหนดค่า

ตัวดำเนินการระดับบิต (Bitwise Operators)

ตัวดำเนินการ	ความหมาย
&	Bitwise AND
	Bitwise OR
^	Bitwise XOR
~	Bitwise NOT

ตัวดำเนินการเชิงตรรกะ (Logical Operators)

ตัวดำเนินการ	ความหมาย
AND	เป็นจริงเมื่อทุกเงื่อนไขที่คั่นด้วย AND เป็นจริง
OR	เป็นจริงเมื่อมีอย่างน้อยหนึ่งเงื่อนไขเป็นจริง
NOT	กลับค่าความจริงของเงื่อนไข
ANY	เป็นจริงเมื่อมีค่าใดค่าหนึ่งจาก subquery ตรงตามเงื่อนไข
ALL	เป็นจริงเมื่อทุกค่าจาก subquery ตรงตามเงื่อนไข

ตัวดำเนินการ	ความหมาย
BETWEEN	เป็นจริงเมื่อค่าตัวดำเนินการอยู่ในช่วงที่กำหนด
LIKE	เป็นจริงเมื่อค่าตรงกับรูปแบบที่กำหนด
IN	เป็นจริงเมื่อค่าตรงกับค่าใดค่าหนึ่งในรายการ

45. คำสั่ง CREATE DATABASE

คำสั่ง CREATE DATABASE ใช้สร้างฐานข้อมูล SQL ใหม่
โดยผู้ใช้ต้องมีสิทธิ์ของผู้ดูแลระบบฐานข้อมูล (Administrator)
จึงจะสามารถสร้างหรือลบฐานข้อมูลได้

รูปแบบคำสั่ง

```
CREATE DATABASE databasename;
```

ตัวอย่าง

สร้างฐานข้อมูลชื่อ testDB:

```
CREATE DATABASE testDB;
```

ตรวจสอบรายชื่อฐานข้อมูล

สำหรับ SQL Server:

```
SELECT name FROM sys.databases;
```

สำหรับ MySQL:

```
SHOW DATABASES;
```

46. คำสั่ง DROP DATABASE

คำสั่ง DROP DATABASE ใช้ลบฐานข้อมูลที่มีอยู่ออกอย่างถาวร
ควรใช้ด้วยความระมัดระวังเป็นอย่างมาก
เนื่องจากจะลบข้อมูลทั้งหมดในฐานข้อมูลนั้นและไม่สามารถกู้คืนได้

รูปแบบคำสั่ง

```
DROP DATABASE databasename;
```

ตัวอย่าง

```
DROP DATABASE testDB;
```

47. คำสั่ง BACKUP DATABASE

คำสั่ง BACKUP DATABASE ใช้ใน SQL Server
เพื่อสำรองข้อมูลฐานข้อมูลทั้งหมดแบบเต็มรูปแบบ (Full Backup)
เก็บไว้ในไฟล์ปลายทางที่กำหนด

รูปแบบคำสั่ง

```
BACKUP DATABASE databasename  
TO DISK = 'filepath';
```

ตัวอย่าง

```
BACKUP DATABASE testDB  
TO DISK = 'D:\\backups\\testDB.bak';
```

การสำรองข้อมูลแบบ Differential

การสำรองแบบ Differential

จะเก็บเฉพาะข้อมูลที่เปลี่ยนแปลงไปนับตั้งแต่การสำรองแบบเต็มรูปแบบครั้งล่าสุด โดยจำเป็นต้องมีการสำรองแบบเต็มรูปแบบมาก่อนอย่างน้อยหนึ่งครั้ง:

```
BACKUP DATABASE testDB
TO DISK = 'D:\\backups\\testDB.bak'
WITH DIFFERENTIAL;
```

48. คำสั่ง CREATE TABLE

คำสั่ง CREATE TABLE ใช้สร้างตารางใหม่ในฐานข้อมูล

รูปแบบคำสั่ง

```
CREATE TABLE table_name (
    column1 datatype constraint,
    column2 datatype constraint,
    column3 datatype constraint,
    ....
);
```

พารามิเตอร์ table_name คือชื่อตารางใหม่ที่ต้องการสร้าง ส่วน datatype คือชนิดข้อมูลของแต่ละคอลัมน์ (เช่น varchar, int, date) และ constraint คือกฎเพิ่มเติมที่กำหนดให้กับคอลัมน์นั้น

ตัวอย่าง

สร้างตารางชื่อ Persons ที่มี 5 คอลัมน์:

```
CREATE TABLE Persons (
    PersonID int,
```

```

LastName varchar(255),
FirstName varchar(255),
Address varchar(255),
City varchar(255)
);

```

สร้างตารางใหม่จากตารางที่มีอยู่แล้ว

สามารถใช้ SELECT INTO หรือ CREATE TABLE ... AS SELECT เพื่อสร้างตารางใหม่พร้อมคัดลอกโครงสร้างและข้อมูลจากตารางเดิม:

```

CREATE TABLE TestTable AS
SELECT customername, contactname
FROM customers;

```

49. คำสั่ง DROP TABLE และ TRUNCATE TABLE

คำสั่ง DROP TABLE ใช้ลบตารางออกจากฐานข้อมูลอย่างถาวร ทั้งโครงสร้างตารางและข้อมูลทั้งหมดภายในจะถูกลบไปด้วย

รูปแบบคำสั่ง DROP TABLE

```
DROP TABLE table_name;
```

ตัวอย่าง

```
DROP TABLE Shippers;
```

คำสั่ง TRUNCATE TABLE

หากต้องการเพียงลบข้อมูลทั้งหมดภายในตารางออก แต่ยังคงเก็บโครงสร้างตารางไว้ ให้ใช้คำสั่ง TRUNCATE TABLE แทน:

```
TRUNCATE TABLE table_name;
```

หมายเหตุ: DROP TABLE ลบทั้งโครงสร้างและข้อมูล ในขณะที่ TRUNCATE TABLE ลบเฉพาะข้อมูลแต่ยังคงโครงสร้างตารางไว้ให้ใช้งานต่อได้

50. คำสั่ง ALTER TABLE

คำสั่ง ALTER TABLE ใช้เพิ่ม ลบ หรือแก้ไขคอลัมน์ในตารางที่มีอยู่แล้ว รวมถึงจัดการข้อจำกัด (Constraint) ต่าง ๆ ของตาราง

เพิ่มคอลัมน์ใหม่

```
ALTER TABLE table_name ADD column_name datatype;
```

ตัวอย่าง: เพิ่มคอลัมน์ Email ในตาราง Customers:

```
ALTER TABLE Customers ADD Email varchar(255);
```

ลบคอลัมน์

```
ALTER TABLE table_name DROP COLUMN column_name;
```

```
ALTER TABLE Customers DROP COLUMN Email;
```

เปลี่ยนชื่อคอลัมน์

```
ALTER TABLE table_name RENAME COLUMN old_name to new_name;
```

แก้ไขชนิดข้อมูลของคอลัมน์

สำหรับ MySQL / Oracle:

```
ALTER TABLE table_name MODIFY COLUMN column_name datatype;
```

เพิ่มข้อจำกัด (Constraint)

```
ALTER TABLE Members
ADD CONSTRAINT CHK_Age CHECK (Age >= 18);
```

เปลี่ยนชื่อตาราง

```
ALTER TABLE table_name RENAME TO new_table_name;
```

ตัวอย่าง: เปลี่ยนชื่อตาราง Customers เป็น Clients:

```
ALTER TABLE Customers RENAME TO Clients;
```

51. ข้อจำกัดของ SQL (SQL Constraints)

ข้อจำกัด (Constraint) คือกฎที่ใช้ควบคุมข้อมูลในตาราง

เพื่อจำกัดประเภทของข้อมูลที่สามารถบันทึกลงในตารางได้

ช่วยรักษาความถูกต้องและความน่าเชื่อถือของข้อมูลในฐานข้อมูล

ข้อจำกัดสามารถกำหนดได้ทั้งตอนสร้างตาราง (CREATE TABLE) หรือภายหลังด้วยคำสั่ง ALTER TABLE

ประเภทของข้อจำกัดที่ใช้งานบ่อย

ข้อจำกัด	ความหมาย
NOT NULL	ห้ามคอลัมน์มีค่าเป็น NULL
UNIQUE	ค่าทุกแถวในคอลัมน์ต้องไม่ซ้ำกัน
PRIMARY KEY	ระบุคีย์หลักที่ใช้ระบุความเป็นเอกลักษณ์ของแต่ละแถว (รวม NOT NULL และ UNIQUE)

ข้อจำกัด	ความหมาย
FOREIGN KEY	เชื่อมโยงข้อมูลระหว่างสองตาราง เพื่อป้องกันการกระทำที่ทำลายความสัมพันธ์ระหว่างตาราง
CHECK	กำหนดเงื่อนไขที่ค่าของคอลัมน์ต้องเป็นจริง
DEFAULT	กำหนดค่าเริ่มต้นให้กับคอลัมน์เมื่อไม่มีการระบุค่า
CREATE INDEX	สร้างดัชนีเพื่อช่วยให้การค้นหาข้อมูลในฐานข้อมูลรวดเร็วขึ้น

52. ข้อจำกัด NOT NULL

ข้อจำกัด NOT NULL บังคับว่าคอลัมน์นั้นจะต้องมีค่าเสมอ ไม่สามารถเว้นว่างหรือมีค่าเป็น NULL ได้ โดยค่าเริ่มต้นของทุกคอลัมน์สามารถมีค่า NULL ได้ เว้นแต่จะระบุข้อจำกัดนี้ไว้

กำหนดตอนสร้างตาราง

```
CREATE TABLE Persons (
  ID int NOT NULL,
  LastName varchar(255) NOT NULL,
  FirstName varchar(255) NOT NULL,
  Age int
);
```

เพิ่มข้อจำกัดในตารางที่มีอยู่แล้ว

สำหรับ SQL Server / MS Access:

```
ALTER TABLE Persons ALTER COLUMN Age int NOT NULL;
```

สำหรับ MySQL:


```
ALTER TABLE Persons MODIFY COLUMN Age int NOT NULL;
```

การยกเลิกข้อจำกัด NOT NULL

สำหรับ SQL Server / MS Access:

```
ALTER TABLE Persons ALTER COLUMN Age int NULL;
```

53. ข้อจำกัด UNIQUE

ข้อจำกัด UNIQUE รับประกันว่าค่าทุกค่าในคอลัมน์ (หรือกลุ่มคอลัมน์) จะไม่ซ้ำกัน ต่างจาก PRIMARY KEY ตรงที่ตารางหนึ่งสามารถมีข้อจำกัด UNIQUE ได้หลายคอลัมน์ แต่มี PRIMARY KEY ได้เพียงหนึ่งเดียว

กำหนดตอนสร้างตาราง

```
CREATE TABLE Persons (
    ID int NOT NULL UNIQUE,
    LastName varchar(255) NOT NULL,
    FirstName varchar(255),
    Age int
);
```

ตั้งชื่อข้อจำกัดและกำหนดกับหลายคอลัมน์

```
CREATE TABLE Persons (
    ID int NOT NULL,
    LastName varchar(255) NOT NULL,
    FirstName varchar(255),
    Age int,
```

```
CONSTRAINT UC_Person UNIQUE (ID, LastName)
);
```

เพิ่มข้อจำกัดในตารางที่มีอยู่แล้ว

```
ALTER TABLE Persons ADD UNIQUE (ID);
```

ยกเลิกข้อจำกัด UNIQUE

สำหรับ MySQL:

```
ALTER TABLE Persons DROP INDEX UC_Person;
```

สำหรับ SQL Server / Oracle / MS Access:

```
ALTER TABLE Persons DROP CONSTRAINT UC_Person;
```

54. ข้อจำกัด PRIMARY KEY

ข้อจำกัด PRIMARY KEY ใช้ระบุความเป็นเอกลักษณ์เฉพาะของแต่ละแถวในตาราง คอลัมน์ที่เป็น PRIMARY KEY จะต้องไม่ซ้ำกันและห้ามเป็นค่า NULL ตารางหนึ่งสามารถมี PRIMARY KEY ได้เพียงหนึ่งเดียวเท่านั้น แต่สามารถประกอบด้วยหลายคอลัมน์รวมกันได้ (composite key)

กำหนดตอนสร้างตาราง

```
CREATE TABLE Persons (
    ID int NOT NULL,
    LastName varchar(255) NOT NULL,
    FirstName varchar(255),
    Age int,
    PRIMARY KEY (ID)
```

```
);
```

PRIMARY KEY จากหลายคอลัมน์

```
CREATE TABLE Persons (
    ID int NOT NULL,
    LastName varchar(255) NOT NULL,
    FirstName varchar(255),
    Age int,
    CONSTRAINT PK_Person PRIMARY KEY (ID, LastName)
);
```

เพิ่ม PRIMARY KEY ในตารางที่มีอยู่แล้ว

```
ALTER TABLE Persons ADD PRIMARY KEY (ID);
```

ยกเลิกข้อจำกัด PRIMARY KEY

สำหรับ SQL Server / Oracle / MS Access:

```
ALTER TABLE Persons DROP CONSTRAINT PK_Person;
```

สำหรับ MySQL:

```
ALTER TABLE Persons DROP PRIMARY KEY;
```

55. ข้อจำกัด FOREIGN KEY

ข้อจำกัด FOREIGN KEY ใช้เชื่อมโยงข้อมูลระหว่างสองตาราง โดยคอลัมน์ในตารางหนึ่ง (ตารางลูก) จะอ้างอิงไปยังคอลัมน์ PRIMARY KEY ของอีกตารางหนึ่ง (ตารางแม่)

เพื่อป้องกันการกระทำที่จะทำให้ลายความสัมพันธ์ระหว่างสองตาราง เช่น การลบข้อมูลในตารางแม่ที่ยังมีข้อมูลอ้างอิงอยู่ในตารางลูก

กำหนดตอนสร้างตาราง

สมมติว่ามีตาราง Persons (ตารางแม่) และตาราง Orders (ตารางลูก) โดยคอลัมน์ PersonID ในตาราง Orders อ้างอิงไปยังคอลัมน์ PersonID ในตาราง Persons:

```
CREATE TABLE Orders (
    OrderID int NOT NULL PRIMARY KEY,
    OrderNumber int NOT NULL,
    PersonID int,
    FOREIGN KEY (PersonID) REFERENCES Persons(PersonID)
);
```

เพิ่ม FOREIGN KEY ในตารางที่มีอยู่แล้ว

```
ALTER TABLE Orders
ADD FOREIGN KEY (PersonID) REFERENCES Persons(PersonID);
```

ยกเลิกข้อจำกัด FOREIGN KEY

สำหรับ SQL Server / Oracle / MS Access:

```
ALTER TABLE Orders DROP CONSTRAINT FK_PersonOrder;
```

สำหรับ MySQL:

```
ALTER TABLE Orders DROP FOREIGN KEY FK_PersonOrder;
```

56. ข้อจำกัด CHECK

ข้อจำกัด CHECK ใช้กำหนดเงื่อนไขที่ค่าของคอลัมน์จะต้องเป็นจริงเสมอ หากมีการพยายามบันทึกค่าที่ไม่ผ่านเงื่อนไขนี้ ระบบจะปฏิเสธการบันทึกข้อมูลนั้น

กำหนดตอนสร้างตาราง

ตัวอย่าง: กำหนดว่าคอลัมน์ Age ต้องมีค่ามากกว่าหรือเท่ากับ 18 เท่านั้น:

```
CREATE TABLE Persons (
  ID int NOT NULL,
  LastName varchar(255) NOT NULL,
  Age int,
  CHECK (Age >= 18)
);
```

ตั้งชื่อและกำหนดกับหลายคอลัมน์

```
CREATE TABLE Persons (
  ID int NOT NULL,
  LastName varchar(255) NOT NULL,
  Age int,
  City varchar(255),
  CONSTRAINT CHK_Person CHECK (Age >= 18 AND City = 'Sandnes')
);
```

เพิ่มข้อจำกัดในตารางที่มีอยู่แล้ว

```
ALTER TABLE Persons ADD CHECK (Age >= 18);
```

ยกเลิกข้อจำกัด CHECK

สำหรับ SQL Server / Oracle / MS Access:

```
ALTER TABLE Persons DROP CONSTRAINT CHK_Person;
```

สำหรับ MySQL:

```
ALTER TABLE Persons DROP CHECK CHK_Person;
```

57. ข้อจำกัด DEFAULT

ข้อจำกัด DEFAULT ใช้กำหนดค่าเริ่มต้นให้กับคอลัมน์

โดยค่าดังกล่าวจะถูกใส่ให้กับทุกระเบียนใหม่โดยอัตโนมัติ หากไม่มีการระบุค่าอื่นตอนเพิ่มข้อมูล

กำหนดตอนสร้างตาราง

```
CREATE TABLE Persons (
    ID int NOT NULL,
    LastName varchar(255) NOT NULL,
    City varchar(255) DEFAULT 'Sandnes'
);
```

ใช้ฟังก์ชันระบบเป็นค่าเริ่มต้น

สามารถใช้ค่า DEFAULT ร่วมกับฟังก์ชันระบบ เช่น การแทรกวันที่ปัจจุบันโดยอัตโนมัติ

ตัวอย่างสำหรับ MySQL:

```
CREATE TABLE Orders (
    ID int NOT NULL,
    OrderDate date DEFAULT CURDATE()
);
```

เพิ่มข้อจำกัด DEFAULT ในตารางที่มีอยู่แล้ว

สำหรับ SQL Server:

```
ALTER TABLE Persons
ADD CONSTRAINT df_City DEFAULT 'Sandnes' FOR City;
```

ยกเลิกข้อจำกัด DEFAULT

สำหรับ SQL Server:

```
ALTER TABLE Persons DROP CONSTRAINT df_City;
```

58. คำสั่ง CREATE INDEX

ดัชนี (Index) ใช้เพื่อค้นหาข้อมูลในฐานข้อมูลได้รวดเร็วยิ่งขึ้น ผู้ใช้งานจะไม่เห็นดัชนีโดยตรง แต่เป็นเบื้องหลังที่ช่วยเร่งความเร็วในการค้นหาหรือคิวรีข้อมูล ดัชนีมีสองประเภทหลัก คือแบบไม่ซ้ำกัน (Non-unique) ซึ่งอนุญาตให้มีค่าซ้ำกันได้ และแบบไม่ซ้ำกันเลย (Unique) ซึ่งบังคับให้ค่าในคอลัมน์ต้องไม่ซ้ำกัน

รูปแบบคำสั่ง CREATE INDEX

```
CREATE INDEX index_name ON table_name (column1, column2, ...);
```

รูปแบบคำสั่ง CREATE UNIQUE INDEX

```
CREATE UNIQUE INDEX index_name ON table_name (column1, column2, ...);
```

ตัวอย่าง

สร้างดัชนีแบบไม่ซ้ำกัน (Non-unique) บนคอลัมน์ LastName ของตาราง Persons:

```
CREATE INDEX idx_lastname ON Persons (LastName);
```

สร้างดัชนีจากหลายคอลัมน์ร่วมกัน:

```
CREATE INDEX idx_pname ON Persons (LastName, FirstName);
```

คำสั่ง DROP INDEX

สำหรับ SQL Server:

```
DROP INDEX table_name.index_name;
```

สำหรับ MySQL:

```
ALTER TABLE table_name DROP INDEX index_name;
```

สำหรับ MS Access:

```
DROP INDEX index_name ON table_name;
```

สำหรับ DB2 / Oracle:

```
DROP INDEX index_name;
```


สรุป

เอกสารฉบับนี้ได้เรียบเรียงเนื้อหาภาษา SQL อย่างครอบคลุม ตั้งแต่แนวคิดพื้นฐานของฐานข้อมูลเชิงสัมพันธ์ คำสั่งดึงข้อมูลด้วย SELECT การกรองข้อมูลด้วย WHERE, AND, OR, NOT, LIKE, IN, BETWEEN การเรียงลำดับด้วย ORDER BY การจัดการข้อมูลด้วย INSERT INTO, UPDATE, DELETE การจัดการค่า NULL (รวมถึง COALESCE, IFNULL, ISNULL, NVL) การจำกัดจำนวนผลลัพธ์ ฟังก์ชันรวมข้อมูล MIN(), MAX(), COUNT(), SUM(), AVG() การใช้นามแฝง (Aliases) การเชื่อมตารางด้วย JOIN ทุกรูปแบบ การรวมผลลัพธ์ด้วย UNION และ UNION ALL การจัดกลุ่มข้อมูลด้วย GROUP BY และ HAVING ตัวดำเนินการกับคำสั้ย่อยอย่าง EXISTS, ANY, ALL การคัดลอกข้อมูลด้วย SELECT INTO และ INSERT INTO SELECT นิพจน์ CASE โปรซีเยอร์ (Stored Procedure) การใช้คอมเมนต์ ตัวดำเนินการประเภทต่าง ๆ

นอกจากนี้ยังครอบคลุมคำสั่งจัดการโครงสร้างฐานข้อมูลและตาราง ได้แก่ CREATE DATABASE, DROP DATABASE, BACKUP DATABASE, CREATE TABLE, DROP TABLE, TRUNCATE TABLE, ALTER TABLE รวมถึงข้อจำกัด (Constraints) ที่สำคัญ ได้แก่ NOT NULL, UNIQUE, PRIMARY KEY, FOREIGN KEY, CHECK, DEFAULT และการสร้างดัชนีด้วย CREATE INDEX เนื้อหาทั้งหมดนี้ครอบคลุมพื้นฐานสำคัญของภาษา SQL ที่ใช้งานจริงในการพัฒนาและดูแลระบบฐานข้อมูลเชิงสัมพันธ์

แหล่งอ้างอิง: <https://www.w3schools.com/sql/>

CTE WITH (คิวรีมีชื่อ)

แนวคิดและการใช้งาน

PostgreSQL: WITH ตั้งชื่อผลลัพธ์ย่อยเพื่อใช้ภายในคำสั่งเดียว ช่วยแบ่งคิวรีซับซ้อนเป็นส่วนย่อย ไม่ใช่การสร้างตารางถาวร

ประเด็นที่ควรเข้าใจ

CTE ช่วยตั้งชื่อส่วนหนึ่งของคิวรี แต่ไม่ได้สั่งให้ฐานข้อมูลเก็บผลถาวรหรือรับประกันว่าประสิทธิภาพจะดีกว่าคิวรีย่อยทุกกรณี ควรใช้เมื่อชื่อและขอบเขตช่วยให้เข้าใจตรรกะได้ง่ายขึ้น

ตัวอย่างสาธิต

```
WITH n AS (SELECT 1 AS value) SELECT value FROM n;
```

ตรวจสอบความเข้าใจ

CTE อยู่ต่อหลังคำสั่งจบหรือไม่?

ไม่อยู่ เป็นชื่อที่ใช้ในขอบเขตคำสั่งนั้น หากต้องการวัดดูเรียกซ้ำข้ามคำสั่งให้ศึกษา view หรือตารางแทน

อ่านเอกสารต้นทาง

<https://www.postgresql.org/docs/current/queries-with.html>

ตรวจ 8 กันยายน 2026 · เรียบเรียงภาษาไทย · ตัวอย่างจัดทำเพื่อสาธิต

Recursive CTE (คิวรีเรียกซ้ำ)

แนวคิดและการใช้งาน

PostgreSQL: WITH RECURSIVE มีส่วนเริ่มต้นและส่วนเรียกซ้ำ เหมาะกับข้อมูลลำดับชั้น ต้องมีเงื่อนไขหยุดหรือป้องกันวงจร

ประเด็นที่ควรเข้าใจ

ส่วนเริ่มต้นเป็นฐานให้การเรียกซ้ำ ส่วน recursive อ่านผลรอบก่อนหน้า การใช้ UNION ALL ไม่ตัดแถวซ้ำ หากข้อมูลเป็นกราฟที่วนกลับได้ต้องออกแบบการตรวจวงจรแยก ไม่ควรใช้คิวรีไม่มีจุดหยุดกับข้อมูลจริง

ตัวอย่างสาธิต

```
WITH RECURSIVE n(x) AS (SELECT 1 UNION ALL SELECT x+1 FROM n WHERE x<3) SELECT x FROM n;
```

ตรวจสอบความเข้าใจ

ตัวอย่าง $n < 3$ หยุดที่ค่าใด?

คืนค่า 1, 2 และ 3 โดยส่วน recursive หยุดสร้างแถวใหม่เมื่อ x มีค่า 3

อ่านเอกสารต้นทาง

<https://www.postgresql.org/docs/current/queries-with.html>

ตรวจ 8 กันยายน 2026 · เรียบเรียงภาษาไทย · ตัวอย่างจัดทำเพื่อสาธิต

Window Functions ROW_NUMBER OVER PARTITION BY (ฟังก์ชันหน้าต่าง)

แนวคิดและการใช้งาน

PostgreSQL: Window function คำนวณข้ามแถวที่เกี่ยวข้องโดยยังเก็บแถวเดิมไว้ ต่างจาก GROUP BY ที่สรุปหลายแถวเป็นกลุ่ม ROW_NUMBER ให้เลขลำดับตาม ORDER BY ภายในหน้าต่าง

ประเด็นที่ควรเข้าใจ

PARTITION BY แบ่งกลุ่มให้ฟังก์ชันทำงานแยกกัน ส่วน ORDER BY ภายใน OVER กำหนดลำดับการคำนวณ ไม่รับประกันลำดับผลลัพธ์สุดท้าย ถ้าต้องการเรียงผลลัพธ์ควรใส่ ORDER BY ของ SELECT ด้วย

ตัวอย่างสาธิต

```
SELECT x, ROW_NUMBER() OVER (ORDER BY x) AS rn FROM (VALUES (20),(10)) AS t(x);
```

ตรวจสอบความเข้าใจ

ROW_NUMBER เหมือน GROUP BY หรือไม่?

ไม่เหมือน Window function เพิ่มผลคำนวณให้แต่ละแถว ส่วน GROUP BY ใช้สรุปแถวเป็นกลุ่ม

อ่านเอกสารต้นทาง

<https://www.postgresql.org/docs/current/tutorial-window.html>

ตรวจ 8 กันยายน 2026 · เรียบเรียงภาษาไทย · ตัวอย่างจัดทำเพื่อสาธิต

Transaction Isolation MVCC (ระดับการแยกธุรกรรม)

แนวคิดและการใช้งาน

PostgreSQL ใช้ Read Committed เป็นค่าเริ่มต้น แต่ละคำสั่งเห็น snapshot เมื่อคำสั่งเริ่ม Repeatable Read ใช้ snapshot ที่คงที่ในธุรกรรม ส่วน Serializable ตรวจสอบความขัดแย้งเพิ่มเติม แอปต้องรองรับการลองธุรกรรมใหม่เมื่อเกิด serialization failure

ประเด็นที่ควรเข้าใจ

ระดับ isolation เป็นสัญญาณการมองเห็นข้อมูล ไม่ใช่แค่ความเร็ว Serializable อาจยกเลิกธุรกรรมที่ขัดกัน แม้แต่ละคำสั่งถูกต้องเมื่อรันเดี่ยว แอปจึงต้อง retry ทั้งธุรกรรมที่ล้มเหลวอย่างเหมาะสม

ตรวจสอบความเข้าใจ

Repeatable Read เท่ากับ Serializable หรือไม่?

ไม่เท่ากัน PostgreSQL Repeatable Read รักษา snapshot แต่ Serializable เพิ่มการตรวจสอบความขัดแย้งเพื่อให้ผลเทียบเท่าการรันตามลำดับ

อ่านเอกสารต้นทาง

<https://www.postgresql.org/docs/current/transaction-iso.html>

ตรวจ 8 กันยายน 2026 · เรียบเรียงภาษาไทย · ตัวอย่างจัดทำเพื่อสาธิต

SQL Injection Prepared Statements (ป้องกันการแทรกคำสั่ง)

แนวคิดและการใช้งาน

แยกข้อความคำสั่ง SQL ออกจากค่าที่ผู้ใช้ป้อนด้วย parameterized queries ห้ามต่อสตริงข้อมูลผู้ใช้เข้า SQL ตรง ๆ ชื่อตารางและคอลัมน์ที่เป็นส่วนโครงสร้างควรเลือกจาก allow-list ไม่ใช้รับข้อความอิสระ ใช้สิทธิ์ฐานข้อมูลเท่าที่จำเป็น รูปแบบ placeholder ขึ้นกับ driver

ประเด็นที่ควรเข้าใจ

ค่าจากผู้ใช้ต้องถูกส่งผ่านช่อง parameter ของ driver ให้ฐานข้อมูลแยกข้อมูลจากคำสั่ง การ escape แบบเขียนเองไม่ใช่แนวทางหลัก ชื่อวัตถุหรือทิศทางเรียงที่แทนด้วย parameter ไม่ได้ให้เลือกรายการที่อนุญาต

ตรวจสอบความเข้าใจ

Prepared statement ป้องกัน input อันตรายได้ทุกส่วนหรือไม่?

การ bind parameter ปกป้องส่วนค่าข้อมูล แต่ไม่ได้ทำให้ชื่อตารางที่ต่อสตริงจากผู้ใช้ปลอดภัย ต้องใช้ allow-list และสิทธิ์ที่จำกัด

อ่านเอกสารต้นทาง

https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html

ตรวจ 8 กันยายน 2026 · เรียบเรียงภาษาไทย · ตัวอย่างจัดทำเพื่อสาธิต

Normalization 1NF 2NF 3NF (การออกแบบลดความซ้ำซ้อน)

แนวคิดและการใช้งาน

1NF จัดข้อมูลไม่ให้มีกลุ่มซ้ำในแถว 2NF แยกข้อมูลที่ขึ้นต่อเพียงบางส่วนของคีย์ประกอบ 3NF แยกข้อมูลที่ขึ้นต่อข้อมูลอื่นซึ่งไม่ใช่คีย์ จุดมุ่งหมายคือช่วยลดความซ้ำซ้อนและปัญหาเมื่อเพิ่ม แก้ไข หรือลบข้อมูล

ประเด็นที่ควรเข้าใจ

แยกข้อมูลที่มีเหตุการณ์เปลี่ยนแปลงต่างกัน เช่น ข้อมูลลูกค้ากับรายการสั่งซื้อ เพื่อลดการแก้ไขข้อมูลซ้ำหลายแถว ความสัมพันธ์ระหว่างตารางต้องยังแทนข้อเท็จจริงทางธุรกิจได้ครบ

ตรวจสอบความเข้าใจ

3NF มีเป้าหมายให้ตารางมากที่สุดหรือไม่?

ไม่มี เป้าหมายคือโครงสร้างความสัมพันธ์ที่ลดความซ้ำซ้อนและปัญหาการปรับข้อมูล ไม่ใช่เพิ่มจำนวนตารางให้มากที่สุด

อ่านเอกสารต้นทาง

<https://learn.microsoft.com/en-us/previous-versions/troubleshoot/microsoft-365/microsoft-365-apps/access/database-normalization-description>

ตรวจ 8 กันยายน 2026 · เรียบเรียงภาษาไทย · ตัวอย่างจัดทำเพื่อสาธิต

EXPLAIN Query Plan (วิเคราะห์แผนคิวรี)

แนวคิดและการใช้งาน

PostgreSQL: EXPLAIN แสดงแผนที่ planner เลือกและต้นทุนประมาณการ EXPLAIN ANALYZE รันคำสั่งจริงเพื่อวัดผล จึงต้องระวังเมื่อใช้กับคำสั่งแก้ข้อมูล ต้นทุนไม่ใช่เวลาเป็นมิลลิวินาทีโดยตรง

ประเด็นที่ควรเข้าใจ

อ่านแผนจากชนิด node จำนวนแถวประมาณ และต้นทุน ก่อนเทียบกับผลจริง การมี sequential scan ไม่ได้หมายความว่าผิดเสมอ เช่นตารางเล็กหรืออ่านข้อมูลส่วนใหญ่ planner อาจเลือกวิธีนี้

ตัวอย่างสาธิต

```
EXPLAIN SELECT 1;
```

ตรวจสอบความเข้าใจ

EXPLAIN ANALYZE เป็นคำสั่งอ่านแผนอย่างเดียวหรือไม่?

ไม่ใช่ ANALYZE รันคำสั่งจริง หากใช้กับคำสั่งที่เปลี่ยนข้อมูลจะเกิดผลข้างเคียงตามคำสั่งนั้น

อ่านเอกสารต้นทาง

<https://www.postgresql.org/docs/current/using-explain.html>

ตรวจ 8 กันยายน 2026 · เรียบเรียงภาษาไทย · ตัวอย่างจัดทำเพื่อสาธิต

Index Performance (ประสิทธิภาพดัชนี)

แนวคิดและการใช้งาน

ดัชนีช่วยค้นแถวเป้าหมายได้เร็วขึ้น แต่มีต้นทุนในการจัดเก็บและดูแลเมื่อแก้ไขข้อมูล ไม่ควรสร้างทุกคอลัมน์โดยไม่วัดผล PostgreSQL มีดัชนีหลายชนิด เช่น B-tree, Hash, GiST, GIN และ BRIN

ประเด็นที่ควรเข้าใจ

การมีดัชนีเป็นทางเลือกให้ planner ไม่ใช่คำสั่งบังคับให้ใช้ ควรเริ่มจากคิวรีที่เป็นปัญหาและวัดผลหลังเพิ่มดัชนี คำสั่งเขียนต้องดูแลดัชนีที่เกี่ยวข้องด้วยจึงมีต้นทุนเพิ่ม

ตรวจสอบความเข้าใจ

เหตุใดไม่สร้างดัชนีทุกคอลัมน์?

ดัชนีใช้พื้นที่และเพิ่มภาระดูแลเมื่อเขียนข้อมูล ประโยชน์ต้องเทียบกับรูปแบบงานจริง

อ่านเอกสารต้นทาง

<https://www.postgresql.org/docs/current/indexes.html>

ตรวจ 8 กันยายน 2026 · เรียบเรียงภาษาไทย · ตัวอย่างจัดทำเพื่อสาธิต

Partitioning (แบ่งตาราง)

แนวคิดและการใช้งาน

PostgreSQL แบ่งตารางเชิงตรรกะเป็นส่วนย่อยทางกายภาพได้แบบ range, list และ hash มีประโยชน์กับตารางใหญ่ และการจัดการข้อมูลเป็นช่วง ต้องเลือก partition key ให้สัมพันธ์กับรูปแบบการใช้งาน ไม่ได้ทำให้ทุกคิวรีเร็วขึ้นโดยอัตโนมัติ

ประเด็นที่ควรเข้าใจ

Partition pruning ช่วยตัดส่วนที่ไม่ต้องอ่านเมื่อเงื่อนไขรองรับ วิธีแบ่งต้องสัมพันธ์กับข้อมูลและคิวรี เช่น แบ่งตามช่วงเวลาเมื่อจัดการข้อมูลตามเดือน ระวัง partition มากเกินไป

ตรวจสอบความเข้าใจ

Partitioning เทียบกับ sharding หรือไม่?

ไม่เท่ากัน บทนี้ว่าด้วยการแบ่งตารางภายใน PostgreSQL ไม่ได้หมายถึงการกระจายฐานข้อมูลไปหลายเครื่องโดยอัตโนมัติ

อ่านเอกสารต้นทาง

<https://www.postgresql.org/docs/current/ddl-partitioning.html>

ตรวจ 8 กันยายน 2026 · เรียบเรียงภาษาไทย · ตัวอย่างจัดทำเพื่อสาธิต

JSON JSONB (ข้อมูลกึ่งโครงสร้าง)

แนวคิดและการใช้งาน

PostgreSQL รองรับฟังก์ชันและตัวดำเนินการอ่าน JSON เครื่องหมาย -> คืบค่า JSON ส่วน ->> คืบข้อความ หากโครงสร้างไม่ตรงหรือไม่มี key ตัวดำเนินการดึงข้อมูลจะคืน SQL NULL

ประเด็นที่ควรเข้าใจ

ชนิดผลลัพธ์สำคัญเมื่อใช้ JSON ต่อในคิวรี -> คืบ JSON ส่วน ->> คืบ text ถ้าต้องคำนวณตัวเลขอาจต้อง cast และจัดการกรณีไม่มี key ข้อมูล JSON null กับ SQL NULL มีความหมายต่างกัน

ตัวอย่างสาริต

```
SELECT '{"name": "Ada"}'::jsonb ->> 'name' AS name;
```

ตรวจสอบความเข้าใจ

ตัวอย่าง ->> 'name' คืบชนิดใด?

คืบ text ชื่อ Ada ไม่ใช่วัตถุ JSON ทั้งก่อน

อ่านเอกสารต้นทาง

<https://www.postgresql.org/docs/current/functions-json.html>

ตรวจ 8 กันยายน 2026 · เรียบเรียงภาษาไทย · ตัวอย่างจัดทำเพื่อสาริต

Backup Restore PITR (สำรองและกู้คืน)

แนวคิดและการใช้งาน

PostgreSQL มีแนวทางสำรองหลักคือ SQL dump, สำรองระดับระบบไฟล์ และ continuous archiving ซึ่งรองรับ point-in-time recovery การเลือกวิธีต้องสัมพันธ์กับเวลาหยุดระบบและจุดเวลาที่ต้องการกู้คืน ควรทดสอบกระบวนการกู้คืนด้วย

ประเด็นที่ควรเข้าใจ

การมีไฟล์สำรองไม่ยืนยันว่ากู้คืนได้ทันเวลาที่ต้องการ วิธี SQL dump เน้นการส่งออกเชิงตรรกะ ส่วน continuous archiving ใช้ข้อมูล WAL ร่วมกับฐานสำรองเพื่อกู้คืนตามเวลา ต้องดูข้อกำหนดแต่ละวิธี

ตรวจสอบความเข้าใจ

สำรองแล้วต้องทดสอบอะไรต่อ?

ทดสอบการ restore ให้เปิดใช้ฐานข้อมูลที่กู้คืนได้ และตรวจสอบว่าช่วงข้อมูลกับเวลาที่ใช้ตรงกับเป้าหมายของงาน

อ่านเอกสารต้นทาง

<https://www.postgresql.org/docs/current/backup.html>

ตรวจ 8 กันยายน 2026 · เรียบเรียงภาษาไทย · ตัวอย่างจัดทำเพื่อสาธิต

Replication Standby (ทำสำเนาฐานข้อมูล)

แนวคิดและการใช้งาน

PostgreSQL streaming replication ส่ง WAL ไป standby มีทั้งแบบ asynchronous และ synchronous แบบ asynchronous อาจสูญเสียธุรกรรมล่าสุดเมื่อ primary ล่มก่อนส่งข้อมูล ส่วน synchronous เพิ่มการรอยืนยันจาก standby ตามการตั้งค่า

ประเด็นที่ควรเข้าใจ

Standby ต้องติดตามการเปลี่ยนแปลงจาก primary การยืนยันแบบ synchronous เพิ่มการรอเพื่อแลกกับความมั่นใจของสำเนาตามการตั้งค่า การเลือก failover และการติดตาม lag เป็นส่วนของการออกแบบระบบ

ตรวจสอบความเข้าใจ

Replication ใช้แทน backup ได้ทั้งหมดหรือไม่?

ไม่ควรถือว่าแทนกัน สำเนาที่ตามการเปลี่ยนแปลงอาจตามการลบผิดไปด้วย ต้องมีแผนสำรองและกู้คืนแยก

อ่านเอกสารต้นทาง

<https://www.postgresql.org/docs/current/warm-standby.html>

ตรวจ 8 กันยายน 2026 · เรียบเรียงภาษาไทย · ตัวอย่างจัดทำเพื่อสาธิต

Row Level Security RLS (สิทธิ์ระดับแถว)

แนวคิดและการใช้งาน

PostgreSQL RLS ใช้นโยบายจำกัดแถวที่ผู้ใช้มองเห็นหรือแก้ไข เมื่อเปิด RLS แต่ไม่มี policy จะเป็น default deny ผู้เป็นเจ้าของตารางมักข้าม RLS ได้ตามค่าเริ่มต้น และ superuser หรือบทบาท BYPASSRLS ข้ามได้ ต้องทดสอบด้วยบทบาทผู้ใช้งานจริง

ประเด็นที่ควรเข้าใจ

RLS เป็นการกรองระดับแถวเพิ่มเติมจากสิทธิ์วัตถุ ต้องกำหนด policy ให้สอดคล้องกับ SELECT และคำสั่งเขียนที่ใช้จริง การทดสอบด้วยเจ้าของตารางอาจไม่เห็นข้อจำกัดแบบเดียวกับบัญชีแอป

ตรวจสอบความเข้าใจ

เปิด RLS แต่ยังไม่มี policy จะเกิดอะไร?

สำหรับบทบาทที่อยู่ภายใต้ RLS จะเป็น default deny ไม่ใช่เปิดให้ทุกคนเห็นทุกแถว

อ่านเอกสารต้นทาง

<https://www.postgresql.org/docs/current/ddl-rowsecurity.html>

ตรวจ 8 กันยายน 2026 · เรียบเรียงภาษาไทย · ตัวอย่างจัดทำเพื่อสาธิต

CREATE VIEW (ตารางเสมือน)

แนวคิดและการทำงาน

PostgreSQL view ปกติเก็บนิยามคิวรี ไม่ได้เก็บผลลัพธ์เป็นสำเนาถาวร เมื่อเรียกใช้จะอ่านผ่านคิวรีที่นิยามไว้ ความสามารถในการแก้ข้อมูลผ่าน view ขึ้นกับรูปแบบคิวรี

ประเด็นที่ควรเข้าใจ

View ทำหน้าที่ให้ชื่อกับคิวรี และช่วยแยกวิธีเรียกข้อมูลออกจากรายละเอียดคิวรีต้นทาง View ปกติไม่ได้เป็น snapshot เก็บผลลัพธ์ การอ่านจึงอิงข้อมูลตามการทำงานของคิวรีในเวลานั้น

ตัวอย่างสาริต

```
CREATE VIEW one_value AS SELECT 1 AS value;
```

ตรวจสอบความเข้าใจ

CREATE VIEW กับ materialized view ต่างกันตรงไหน?

View ปกติไม่เก็บผลลัพธ์ถาวร ส่วน materialized view เก็บผลและต้อง refresh เพื่อปรับข้อมูลให้ใหม่

อ่านเอกสารต้นทาง

<https://www.postgresql.org/docs/current/sql-createview.html>

ตรวจ 8 กันยายน 2026 · เรียบเรียงภาษาไทย · ตัวอย่างจัดทำเพื่อสาริต

Identity AUTO INCREMENT (เลขอัตโนมัติ)

แนวคิดและการใช้งาน

PostgreSQL ใช้ GENERATED ALWAYS หรือ BY DEFAULT AS IDENTITY เพื่อให้ implicit sequence สร้างค่าอัตโนมัติ Identity ไม่ได้บังคับความไม่ซ้ำด้วยตัวเอง ต้องเพิ่ม PRIMARY KEY หรือ UNIQUE หากต้องการ ความสามารถนี้ไม่ใช่ไวยากรณ์ AUTO_INCREMENT ของ MySQL

ประเด็นที่ควรเข้าใจ

GENERATED ALWAYS ควบคุมการใส่ค่าเองเข้มกว่า BY DEFAULT ซึ่งยอมให้ระบุค่าแทนค่าจาก sequence ในการใช้งานทั่วไป ระบบเลขอัตโนมัติไม่ได้สัญญาว่าจะต่อเนื่องไม่มีช่องว่าง

ตัวอย่างสาธิต

```
CREATE TABLE identity_demo (id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY);
```

ตรวจสอบความเข้าใจ

Identity ทำให้คอลัมน์ unique โดยอัตโนมัติหรือไม่?

ไม่ การรับประกันความไม่ซ้ำต้องมี PRIMARY KEY หรือ UNIQUE ตามโครงสร้างที่ต้องการ

อ่านเอกสารต้นทาง

<https://www.postgresql.org/docs/current/ddl-identity-columns.html>

ตรวจ 8 กันยายน 2026 · เรียบเรียงภาษาไทย · ตัวอย่างจัดทำเพื่อสาธิต

Subquery EXISTS NOT IN NULL (คิวรีย่อยและค่าว่าง)

แนวคิดและการใช้งาน

PostgreSQL: EXISTS เป็นจริงเมื่อคิวรีย่อยคืนอย่างน้อยหนึ่งแถว NOT IN อาจได้ NULL แทนจริงเมื่อฟังก์คิวรีย่อยมี NULL และไม่มีค่าที่ตรงกัน ต้องพิจารณาพฤติกรรม NULL ก่อนใช้กรองข้อมูล

ประเด็นที่ควรเข้าใจ

คิวรีย่อยเป็นคำสั่งภายในอีกคำสั่ง EXISTS สนใจว่ามีแถวหรือไม่ ไม่ได้สนใจค่าที่ SELECT คืนทั้งหมด ส่วน NOT IN ต้องระวังตรรกะสามค่าเมื่อพบ NULL

ตัวอย่างสาธิต

```
SELECT EXISTS (SELECT 1 WHERE 1=1) AS found;
```

ตรวจสอบความเข้าใจ

NOT IN ที่มี NULL ทางขวาให้ true เสมอเมื่อไม่มีค่าตรงหรือไม่?

ไม่เสมอ อาจให้ NULL ทำให้แถวไม่ผ่าน WHERE ต้องพิจารณา NULL ในข้อมูลและเลือกเงื่อนไขให้ตรงความหมาย

อ่านเอกสารต้นทาง

<https://www.postgresql.org/docs/current/functions-subquery.html>

ตรวจ 8 กันยายน 2026 · เรียบเรียงภาษาไทย · ตัวอย่างจัดทำเพื่อสาธิต

Transaction BEGIN COMMIT ROLLBACK SAVEPOINT (ธุรกรรมและจุดย้อนกลับ)

แนวคิดและการใช้งาน

ธุรกรรมรวมหลายคำสั่งให้เป็นหน่วยงานเดียวที่สำเร็จทั้งหมดหรือยกเลิกทั้งหมด BEGIN เริ่มกลุ่มงาน COMMIT ยืนยัน และ ROLLBACK ยกเลิกการเปลี่ยนแปลงในกลุ่มงาน PostgreSQL ทำงานในธุรกรรมแม้ไม่ได้เขียน BEGIN เอง โดย driver อาจควบคุม autocommit ให้

ประเด็นที่ควรเข้าใจ

SAVEPOINT สร้างจุดย้อนกลับภายในธุรกรรม ROLLBACK TO ย้อนเฉพาะส่วนหลังจุดนั้น ส่วนก่อนหน้ายังคงรอ COMMIT การมี savepoint ไม่ได้ทำให้ข้อมูลถูกยืนยันเป็นอิสระจากธุรกรรมหลัก

ตัวอย่างสาธิต

```
BEGIN;  
SAVEPOINT draft;  
SELECT 1;  
ROLLBACK TO draft;  
COMMIT;
```

ตรวจสอบความเข้าใจ

ROLLBACK TO ต่างจาก ROLLBACK อย่างไร?

ROLLBACK TO ย้อนถึง savepoint ที่ระบุ ส่วน ROLLBACK ยกเลิกทั้งธุรกรรม ตัวอย่างนี้มีแต่ SELECT จึงสาธิตการควบคุมธุรกรรม ไม่ได้เปลี่ยนข้อมูลจริง

อ่านเอกสารต้นทาง

<https://www.postgresql.org/docs/current/tutorial-transactions.html>

ตรวจ 8 กันยายน 2026 · เรียบเรียงภาษาไทย · ตัวอย่างจัดทำเพื่อสาธิต

Locking Deadlock (การล็อกและเดดล็อก)

แนวคิดและการใช้งาน

PostgreSQL ใช้ lock ควบคุมการเข้าถึงข้อมูลพร้อมกัน มีทั้งระดับตารางและแถว คำสั่งหลายชนิดขอ lock อัตโนมัติ `SELECT FOR UPDATE` ใช้ล็อกแถวที่เลือกเพื่อแก้ไขภายหลังในธุรกรรมเดียวกัน

ประเด็นที่ควรเข้าใจ

Deadlock เกิดเมื่อธุรกรรมต่างรอทรัพยากรที่อีกฝ่ายถืออยู่ PostgreSQL ตรวจพบและยกเลิกหนึ่งธุรกรรมเพื่อให้ระบบเดินต่อไปได้ การเข้าถึงทรัพยากรตามลำดับเดียวกันช่วยลดโอกาสเกิด และแอปควรรองรับ retry

ตัวอย่างสาริต

```
SELECT locktype, mode, granted
FROM pg_locks
LIMIT 10;
```

ตรวจสอบความเข้าใจ

การเห็น `granted` เป็น `false` แปลว่าอะไร?

เป็น lock ที่ยังไม่ได้รับอนุญาตและกำลังรอ ไม่ได้ยืนยันว่าเป็น deadlock เสมอไป ต้องดูธุรกรรมและความสัมพันธ์ของการรอประกอบ

อ่านเอกสารต้นทาง

<https://www.postgresql.org/docs/current/explicit-locking.html>

ตรวจ 8 กันยายน 2026 · เรียบเรียงภาษาไทย · ตัวอย่างจัดทำเพื่อสาธิต

Roles GRANT REVOKE (บทบาทและสิทธิ์)

แนวคิดและการใช้งาน

PostgreSQL ใช้ role แทนทั้งผู้ใช้และกลุ่ม ผู้ใช้ที่เชื่อมต่อได้ต้องมีคุณสมบัติ LOGIN และสามารถเป็นสมาชิกของ role อื่นเพื่อรับสิทธิ์ได้ แยกบทบาทสำหรับเจ้าของวัตถุ ผู้ดูแล และแอปพลิเคชันให้เหมาะกับงาน

ประเด็นที่ควรเข้าใจ

GRANT และ REVOKE ใช้จัดการสิทธิ์ในวัตถุและสมาชิกของ role การมี role ไม่ได้แปลว่าอ่านทุกตารางได้ ควรทดสอบการทำงานภายใต้ role ของแอปจริง ไม่ใช่ superuser เป็นตัวแทน

ตัวอย่างสาริต

```
SELECT current_user;
```

ตรวจสอบความเข้าใจ

เหตุใดการทดสอบด้วยผู้ดูแลจึงไม่ยืนยันสิทธิ์ของแอป?

ผู้ดูแลอาจมีสิทธิ์มากกว่าบัญชีแอป คิวรีที่รันผ่านภายใต้ผู้ดูแลจึงอาจถูกปฏิเสธเมื่อใช้บัญชีจริง

อ่านเอกสารต้นทาง

<https://www.postgresql.org/docs/current/user-manag.html>

ตรวจ 8 กันยายน 2026 · เรียบเรียงภาษาไทย · ตัวอย่างจัดทำเพื่อสาริต

Date Time Timestamp Timezone (วันเวลาและเขตเวลา)

แนวคิดและการใช้งาน

PostgreSQL มี date, time, timestamp และ interval โดย timestamp without time zone ไม่เก็บความหมายของเขตเวลา ส่วน timestamp with time zone หรือ timestamptz แปลงเวลาไปเก็บเป็น UTC และแสดงตามเขตเวลาของ session ไม่ได้รักษาชื่อเขตเวลาดั้งเดิมไว้

ประเด็นที่ควรเข้าใจ

ควรแยกวันที่ปฏิทินออกจากเวลาของเหตุการณ์จริง การระบุ offset ชัดเจนช่วยหลีกเลี่ยงความกำกวมเมื่อแลกข้อมูลข้ามประเทศ ค่าที่แสดงจาก timestamptz อาจต่างกันตาม session แม้เป็นเหตุการณ์เดียวกัน

ตัวอย่างสาธิต

```
SELECT DATE '2026-09-08' AS day,  
       TIMESTAMPTZ '2026-09-08 09:00:00+07'  
       AS happened_at;
```

ตรวจสอบความเข้าใจ

ต้องเก็บชื่อ Asia/Bangkok ไว้หรือไม่?

หากงานต้องรักษาชื่อเขตเวลาดั้งเดิม ควรเก็บข้อมูลชื่อนั้นแยก เพราะ timestamptz เก็บจุดเวลา ไม่ใช่ชื่อเขตเวลาที่ป้อน

อ่านเอกสารต้นทาง

<https://www.postgresql.org/docs/current/datatype-datetime.html>

ตรวจ 8 กันยายน 2026 · เรียบเรียงภาษาไทย · ตัวอย่างจัดทำเพื่อสาธิต

Materialized View REFRESH (ผลลัพธ์ที่จัดเก็บไว้)

แนวคิดและการใช้งาน

Materialized view เก็บผลลัพธ์ควิรีไว้ทางกายภาพ ต่างจาก view ปกติที่อ่านผ่านคิวรีนิยาม เมื่อข้อมูลต้นทางเปลี่ยน ผลลัพธ์ที่เก็บจะไม่สดใหม่จนกว่าจะ refresh

ประเด็นที่ควรเข้าใจ

เหมาะกับการอ่านผลสรุปซ้ำที่ยอมรับความล่าช้าของข้อมูลได้ ต้องกำหนดว่าจะ refresh เมื่อใดและให้ผู้ใช้เข้าใจเวลาของข้อมูล ไม่ควรถือว่าผลสรุปเป็นข้อมูลสดตลอดเวลา

ตัวอย่างสาธิต

```
CREATE MATERIALIZED VIEW demo_snapshot AS
SELECT 1 AS value;
REFRESH MATERIALIZED VIEW demo_snapshot;
```

ตรวจสอบความเข้าใจ

ข้อแลกเปลี่ยนเมื่อใช้ materialized view คืออะไร?

อ่านผลที่จัดเก็บไว้ได้โดยไม่คำนวณคิวรีเดิมทุกครั้ง แต่ต้องใช้พื้นที่และจัดการการ refresh ตัวอย่างนี้สร้างวัตถุในฐานทดลอง

อ่านเอกสารต้นทาง

<https://www.postgresql.org/docs/current/rules-materializedviews.html>

ตรวจ 8 กันยายน 2026 · เรียบเรียงภาษาไทย · ตัวอย่างจัดทำเพื่อสาธิต

Trigger BEFORE AFTER (การทำงานตามเหตุการณ์)

แนวคิดและการใช้งาน

Trigger สั่งให้ฟังก์ชันทำงานเมื่อเกิดเหตุการณ์บนตารางหรือ view PostgreSQL รองรับการทำงาน BEFORE, AFTER และบางกรณี INSTEAD OF โดยแยกได้ทั้งต่อแถวและต่อคำสั่ง

ประเด็นที่ควรเข้าใจ

Trigger ต่อแถวทำงานตามจำนวนแถวที่ได้รับผล ส่วนต่อคำสั่งทำงานต่อคำสั่งแม้ไม่มีแถวเปลี่ยน ต้องคำนึงถึงผลข้างเคียงและการเรียกซ้ำเมื่อฟังก์ชัน trigger แก่ตารางที่มี trigger อีก

ตรวจสอบความเข้าใจ

คำสั่ง UPDATE ที่แก้ 100 แถวทำให้ trigger ทำงานกี่ครั้ง?

ขึ้นกับระดับที่กำหนด: trigger แบบ FOR EACH ROW ทำงานกับแต่ละแถว ส่วนแบบ FOR EACH STATEMENT ทำงานระดับคำสั่ง ไม่สามารถสรุปจำนวนครั้งจาก UPDATE อย่างเดียว

อ่านเอกสารต้นทาง

<https://www.postgresql.org/docs/current/trigger-definition.html>

ตรวจ 8 กันยายน 2026 · เรียบเรียงภาษาไทย · ตัวอย่างจัดทำเพื่อสาธิต

UPSERT ON CONFLICT (เพิ่มหรือปรับข้อมูลเมื่อชนคีย์)

แนวคิดและการใช้งาน

PostgreSQL INSERT ... ON CONFLICT ใช้จัดการความขัดแย้งของข้อจำกัด unique ที่เกี่ยวข้อง DO NOTHING ข้ามแถวที่ชน ส่วน DO UPDATE ระบุการแก้แถวเดิม EXCLUDED ใช้อ้างถึงค่าของแถวที่พยายามเพิ่ม

ประเด็นที่ควรเข้าใจ

การเลือก conflict target ต้องสัมพันธ์กับ unique constraint หรือดัชนีที่รองรับ ไม่ใช่กลไกแก้ error ทุกชนิด และไม่ได้ทำให้ข้อมูลที่ละเมิดเงื่อนไขอื่นผ่านได้

ตัวอย่างสาริต

```
CREATE TEMP TABLE demo_upsert (  
  id integer PRIMARY KEY, label text  
);  
INSERT INTO demo_upsert VALUES (1, 'first');  
INSERT INTO demo_upsert VALUES (1, 'updated')  
ON CONFLICT (id) DO UPDATE  
SET label = EXCLUDED.label;
```

ตรวจสอบความเข้าใจ

หลังตัวอย่างจะมีข้อมูล id=1 กี่แถว?

มีหนึ่งแถว โดย label เป็น updated เพราะ PRIMARY KEY ระบุความขัดแย้งและ DO UPDATE แก้แถวเดิม ตัวอย่างใช้ตารางชั่วคราว

อ่านเอกสารต้นทาง

<https://www.postgresql.org/docs/current/sql-insert.html>

ตรวจ 8 กันยายน 2026 · เรียบเรียงภาษาไทย · ตัวอย่างจัดทำเพื่อสาธิต

NoSQL Document Modeling (ออกแบบฐานข้อมูลเอกสาร)

แนวคิดและการใช้งาน

MongoDB จัดเก็บข้อมูลเป็นเอกสารและสนับสนุน schema ที่ยืดหยุ่น การออกแบบควรเริ่มจากรูปแบบการอ่านและเขียน แล้วเลือกระหว่างฝังข้อมูลที่เกี่ยวข้องในเอกสารเดียวกับใช้การอ้างอิงข้ามเอกสาร

ประเด็นที่ควรเข้าใจ

ข้อมูลที่อ่านร่วมกันบ่อยอาจเหมาะกับ embedding แต่ต้องพิจารณาขนาดเอกสารและการเติบโต การใช้ references เหมาะเมื่อข้อมูลมีการใช้ร่วมกันหรือมีความสัมพันธ์ที่เปลี่ยนได้ ไม่ควรแปลว่า NoSQL ไม่ต้องออกแบบ schema

ตรวจสอบความเข้าใจ

ต้องเลือกระหว่าง SQL และ NoSQL ด้วยเกณฑ์ใด?

สำหรับเนื้อหาบทนี้ให้พิจารณารูปแบบเข้าถึงข้อมูล ความสัมพันธ์ และการฝังหรืออ้างอิงข้อมูล ไม่ควรสรุปว่าระบบหนึ่งเร็วกว่าหรือดีกว่าทุกกรณีจากชนิดฐานข้อมูลเพียงอย่างเดียว

อ่านเอกสารต้นทาง

<https://www.mongodb.com/docs/manual/data-modeling/>

ตรวจ 8 กันยายน 2026 · เรียบเรียงภาษาไทย · ตัวอย่างจัดทำเพื่อสาธิต

Sharding Shard Key (กระจายข้อมูลข้ามเครื่อง)

แนวคิดและการใช้งาน

MongoDB sharding กระจายข้อมูลข้ามหลายเครื่องเพื่อรองรับชุดข้อมูลและปริมาณงานที่เพิ่มขึ้น shard key เป็นตัวกำหนดการกระจายข้อมูล และส่งผลต่อประสิทธิภาพการกำหนดเส้นทางคิวรี

ประเด็นที่ควรเข้าใจ

การกระจายข้อมูลต่างจาก replication ซึ่งทำสำเนาข้อมูล คิวรีที่ไม่มีข้อมูลช่วยเลือก shard อาจต้องกระจายไปหลาย shard จึงต้องเลือก key จากรูปแบบการอ่านเขียนและการกระจายของค่า

ตรวจสอบความเข้าใจ

เพิ่ม shard แล้วทุกคิวรีจะเร็วขึ้นหรือไม่?

ไม่จำเป็น ประโยชน์ขึ้นกับ shard key รูปแบบคิวรี และการกระจายงาน ข้อมูลที่กระจุกอยู่เครื่องเดียวหรือคิวรีที่ต้องอ่านทุก shard ยังอาจเป็นคอขวดได้

อ่านเอกสารต้นทาง

<https://www.mongodb.com/docs/manual/sharding/>

ตรวจ 8 กันยายน 2026 · เรียบเรียงภาษาไทย · ตัวอย่างจัดทำเพื่อสาธิต

Schema Migration ALTER TABLE (เปลี่ยนโครงสร้างข้อมูล)

แนวคิดและการใช้งาน

PostgreSQL ALTER TABLE ใช้เพิ่มหรือลบคอลัมน์ เปลี่ยนชนิดข้อมูล ปรับค่าเริ่มต้น และเพิ่มข้อจำกัด แต่ละการเปลี่ยนมีผลต่อข้อมูลเดิมต่างกัน เช่น การเปลี่ยนชนิดต้องแปลงค่าเดิมได้

ประเด็นที่ควรเข้าใจ

ก่อนย้าย schema ควรตรวจสอบข้อมูลที่มีอยู่และการพึ่งพาจากแอป การเพิ่ม constraint อาจล้มเหลวเมื่อข้อมูลเก่าละเมิดเงื่อนไข การตั้ง DEFAULT ใหม่ใช้กับการเพิ่มข้อมูลภายหลัง ไม่ได้แก้ค่าที่มีอยู่ทุกแถวให้เอง

ตัวอย่างสาริต

```
CREATE TEMP TABLE demo_migration (id integer);  
ALTER TABLE demo_migration  
ADD COLUMN note text DEFAULT 'new';
```

ตรวจสอบความเข้าใจ

ตั้ง **DEFAULT** หลังมีข้อมูลแล้ว เท่ากับ **UPDATE** ทุกแถวหรือไม่?

ไม่เท่ากัน การแก้ default เปลี่ยนค่าที่ใช้เมื่อเพิ่มข้อมูลใหม่โดยไม่ระบุค่า ไม่ใช่คำสั่งปรับข้อมูลเดิมทั้งหมด

อ่านเอกสารต้นทาง

<https://www.postgresql.org/docs/current/ddl-alter.html>

ตรวจ 8 กันยายน 2026 · เรียบเรียงภาษาไทย · ตัวอย่างจัดทำเพื่อสาริต

บรรณานุกรมและที่มาของข้อมูล

ต้นฉบับ 72 หน้าจัดเตรียมโดยผู้ใช้ และระบุ W3Schools บนหน้าปก ภาคเสริมใช้เอกสาร PostgreSQL, Microsoft Learn, OWASP และ MongoDB ด้านล่าง วันที่ตรวจเอกสาร 8 กันยายน 2026

01. <https://www.w3schools.com/sql/>
02. <https://www.postgresql.org/docs/current/queries-with.html>
03. <https://www.postgresql.org/docs/current/tutorial-window.html>
04. <https://www.postgresql.org/docs/current/transaction-iso.html>
05. https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html
06. <https://learn.microsoft.com/en-us/previous-versions/troubleshoot/microsoft-365/microsoft-365-apps/access/database-normalization-description>
07. <https://www.postgresql.org/docs/current/using-explain.html>
08. <https://www.postgresql.org/docs/current/indexes.html>
09. <https://www.postgresql.org/docs/current/ddl-partitioning.html>
10. <https://www.postgresql.org/docs/current/functions-json.html>
11. <https://www.postgresql.org/docs/current/backup.html>
12. <https://www.postgresql.org/docs/current/warm-standby.html>
13. <https://www.postgresql.org/docs/current/ddl-rowsecurity.html>
14. <https://www.postgresql.org/docs/current/sql-createview.html>
15. <https://www.postgresql.org/docs/current/ddl-identity-columns.html>
16. <https://www.postgresql.org/docs/current/functions-subquery.html>
17. <https://www.postgresql.org/docs/current/tutorial-transactions.html>
18. <https://www.postgresql.org/docs/current/explicit-locking.html>
19. <https://www.postgresql.org/docs/current/user-manag.html>
20. <https://www.postgresql.org/docs/current/datatype-datetime.html>
21. <https://www.postgresql.org/docs/current/rules-materializedviews.html>
22. <https://www.postgresql.org/docs/current/trigger-definition.html>
23. <https://www.postgresql.org/docs/current/sql-insert.html>
24. <https://www.mongodb.com/docs/manual/data-modeling/>
25. <https://www.mongodb.com/docs/manual/sharding/>
26. <https://www.postgresql.org/docs/current/ddl-alter.html>

เก็บข้อมูลเก่าและใหม่แยกที่มา การรวมเอกสารไม่ได้เป็นการรับรองความถูกต้องของข้อความต้นฉบับทุกบรรทัด ควรตรวจสอบเอกสารของฐานข้อมูลที่ใช้งานจริงก่อนรันคำสั่ง